

Prediction-Driven Non-Preemptive Scheduling with Bounded Overtaking for Shared Execution Services under Deadline-Driven Bursty Load

[First Author Name]¹ and [Second Author Name]^{2,*}

¹ [First author: department, institution, city and postcode, country]; [first author e-mail]

² [Second author: department, institution, city and postcode, country]; [second author e-mail]

* Correspondence: [corresponding author e-mail]

Abstract

A shared execution service runs jobs on a fixed pool of servers, without preemption, under a per-job time limit L . Reordering by predicted cost shortens most waits and can leave one job far behind. We prove a pathwise identity: for any non-preemptive work-conserving k -server policy and input, a job's delay relative to first-come first-served equals its net overtaken work divided by k , to within $(2 - 2/k)L$, an exact supremum. Bounding overtaken work is therefore sufficient for a per-job guarantee, and necessary up to an additive $O(kL)$; a count of overtakes is not necessary, and suffices only by charging L per pass. A wrapper enforcing that budget bounds every wait against first-come first-served under arbitrary arrivals and predictions. It reads the budget only through a cap, so the shape below is free; the additive constant $(3 - 2/k)L$ is likewise exact, and the cap follows from a promise in seconds. Expected-cost ordering is an empirically motivated base policy rather than a multiserver optimality claim. We evaluate on two judging platforms, a serverless trace, two continuous-integration pools that check the simulator, and a compute farm. The empirical headline uses conservative outcome visibility: at the busiest development load, a 600-second guard closes 0.429 of the FCFS-to-SJF tail-wait gap, with interval $[0.357, 0.533]$. Original-clock results are optimistic references. We report when the uniform certificate is too coarse for the observed waits and when cost concentration offers little benefit.

Keywords: non-preemptive scheduling; scheduling with predictions; multiserver queues; bounded overtaking; per-job guarantee; first-come first-served; runtime prediction

MSC: 90B35; 68M20; 60K25; 68W27

1. Introduction

A shared execution service runs jobs on behalf of many users on a fixed pool of k servers. A job that has started is not interrupted: it runs until it finishes or until it reaches a per-job time limit L . Serverless function platforms, build and test farms for continuous integration, batch inference queues, and the judging back ends of programming-assignment platforms are all organised this way [1]. Operators document the consequence themselves: a contest judging system recommends one judging host per twenty teams and notes that with fewer hosts the waiting time grows [2], and the maintainers of a university autograder replayed the busiest four hours before a common deadline and found that one submission looping until its resource limit took a grader out of circulation and delayed the graders behind it [3]; their replay compresses that window, a method we argue against in

Received:

Revised:

Accepted:

Published:

Copyright: © 2026 by the authors.

Submitted to *Mathematics* for possible open access publication under the terms and conditions of the [Creative Commons Attribution \(CC BY\)](#) license.

Section 9 and replace by overlaying whole class-terms. Two properties of the offered load make such a service slow down when it is used most. Arrivals follow an external calendar, so they are neither stationary nor independent; the same concentration near a fixed cutoff has been measured directly [4–6]. And job costs are concentrated in a small fraction of jobs: on the serverless invocation trace we use [7,8], the longest 1% of invocations account for 44% of the total work, and on the main judging trace, after the analysis service cap is applied, the longest 1% still account for 37.5%, and 27.5% on the second. A few very long jobs therefore sit in front of many short ones exactly when the queue is longest.

The mechanism we exploit is visible in a queue of three jobs on one server (Figure 1). Job *A* loops forever and is stopped by the 60-second limit; jobs *B* and *C* take 0.2 seconds each. In arrival order the mean wait is 40.1 seconds; shortest first it is 0.2 seconds, at a cost of 0.4 seconds to *A*. The server performs the same total work in both orders. The three numbers are arithmetic; we have not simulated anything yet, and they rest on one assumption: that the scheduler knows, before *A* starts, that *A* will be long. That assumption raises three questions, and the paper is organised around them. Can the cost of a job be estimated at the moment it arrives, from history that is causally visible at that instant? Given estimates, what should a non-preemptive scheduler rank by? And what happens to an individual job when the estimates are wrong?

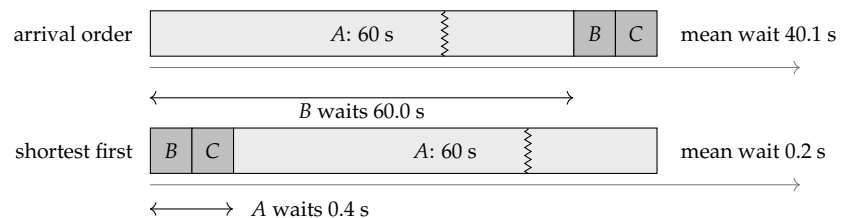


Figure 1. The three-job example of Section 1 on one server: *A* is stopped at the per-job limit $L = 60$ s, *B* and *C* take 0.2 s each, and all three are present at $t = 0$. The boxes are schematic, not to scale, and the zigzag marks the elided part of *A*. The two orders execute the same total work; reordering moves 60 seconds of waiting off *B* and *C* and puts 0.4 seconds onto *A*. Source: the arithmetic stated in the text, not a measurement.

On the first, arrival-time history is enough to tell the heavy jobs from the rest on all three platforms. We compare three families of predictor on one split and one visibility protocol — tabular models [9], a sequence model over each user’s history, and a heterogeneous graph neural network over the user, task and assignment entities [10] — and the graph model does not improve on the tabular one: removing every edge, and then permuting the edges at random, leaves its classification scores where they were, which places the signal in each entity’s own history rather than in the relational structure. We report that as a property of this prediction task and not of relational models (Section 4). On the second question, we rank by estimated conditional expected cost $\mathbb{E}[C \mid x]$ rather than by a log-scale prediction, whose back-transform generally differs from the arithmetic mean; the single-server interchange argument motivates that choice without establishing optimality for multiple servers, arbitrary arrivals or deadline-window percentiles. Errors in the fitted score are asymmetric, treating a heavy job as light and letting it run first being the expensive mistake by about an order of magnitude, which diagnoses that score rather than establishing a universal loss-function comparison (Sections 5 and 8.4).

The third question is the one an operator will ask first. Ordering by prediction improves the bulk of the distribution and can damage the tail: in our busiest configuration one job waited almost a hundred job-limits longer than it would have under first-come first-served (Section 8.5). Robustness results in the scheduling-with-predictions literature usually bound an expectation against an optimal or shortest-remaining-processing-time reference, and the constant-factor ones require the prediction error to be bounded multi-

plicatively, which is not a property anyone can promise about a deployed predictor [11,12]. There are exceptions on single axes. One line bounds the maximum stretch over all jobs on a preemptive machine, which is not an expectation [13]; another bounds a request's latency in a stationary light-tailed queue against the workload the system holds at the request's boosted arrival instant, using the work that overtakes it [14,15]; read on a sample path that is close to a bound against first-come first-served, but it is not the form those papers state. What we do not find is the conjunction: a first-come first-served reference, stated per job, on an arbitrary arrival sequence with arbitrary predictions, non-preemptive, on k servers, and enforced online around whatever policy is underneath instead of analysed for one fixed policy. We change the reference point instead of strengthening the assumption, and bound, for each job separately, how much longer it waits than it would have waited under first-come first-served on the same arrival sequence, assuming nothing about the arrival process and nothing about prediction quality. The mechanism is an overtake budget: for every waiting job we account the total work of later-ordered jobs that have completed while it waited, and at each dispatch decision the smallest-ranked waiting job whose account reaches its current budget takes precedence. The guard wraps an arbitrary base scheduler, including a learned one, and touches only the dispatch decision. It is one mechanism with one parameter that matters for the promise, a cap $B_{\max}$ on the budget; what the budget does below that cap is a design freedom that changes how much of the base policy's benefit survives and which jobs pay for it, and changes the promise not at all.

We budget work rather than count overtakes because the identity puts the guarantee in units of work: a job's delay relative to first-come first-served equals its net overtaken work divided by k , to within $(2 - 2/k)L$, for every non-preemptive work-conserving k -server policy and every arrival sequence. The identity does not refer to the policy and it cuts both ways, which is what makes a work budget the right currency and a bound on the *number* of overtakes the wrong one. The guarantee is denominated in units of L , so the uniform certificate is most informative when L is comparable to the waits an operator cares about; a computing farm whose jobs run for days illustrates a certificate too coarse for those waits, which does not rule out tighter guarantees for a restricted policy or workload.

We make four contributions.

- The net-overtake identity just stated, with the constant $(2 - 2/k)L$ shown to be an exact supremum, approached and never attained, and a corollary in the other direction: any per-job guarantee of this kind bounds the overtaken work, up to an additive $O(kL)$.
- An overtake-budget guard. It wraps any base scheduler and bounds every job's wait against first-come first-served, for k servers, arbitrary arrivals and arbitrary predictions. The bound reads the budget only through a cap: any budget rule with $0 \leq \text{budget} \leq B_{\max}$ gives the same guarantee, so the operator sets $B_{\max} = k(G - (3 - 2/k)L)$ from a promise G in seconds, with no data, and is then free in how the budget is spent below it. The additive constant $(3 - 2/k)L$, beyond the budget term, is an exact supremum, never attained, and it grows with k from L at $k = 1$ towards $3L$; no analysis of this mechanism can improve it. One shape of budget, the one whose allowance grows with the wait a job has already had, carries a multiplicative bound as well. We state which assumptions can be dropped, and show by counterexample that work conservation cannot.
- A comparison of cost predictors on causally visible history, covering tabular, sequence and heterogeneous graph models, with two conclusions that carry into the scheduler: expected-cost ordering performs better than the log-scale alternative in the reported source-clock comparisons, and the fitted score is more sensitive to severe

underestimates. Separate conservative and static controls examine the availability of that history in a replayed queue.

- An evaluation on real logs from two programming-assignment judging platforms [16,17] and a serverless invocation trace [7], with every simulated job checked against the theoretical bound, and two applicability conditions read off six workloads, most of which fail one or the other: the uniform certificate is too coarse where the waits are short relative to L , and there is little to exploit where costs are not concentrated [18–21]. Section 8.8 reads both conditions off the six in numbers and Section 9 states what they rule out.

Four qualifications hold over everything reported below, and each is stated where the numbers are. The judging logs supply recorded execution demand and do not identify a historical shared queue, so the pooled capacity and the replayed waiting times are experimental constructions of ours (Section 7); a capacity sweep and an executing timed-work service test the mechanism under fixed offered work, and the physical guard, while it follows the dispatch rule, has a higher p99 than the separately timed first-come first-served run and does not reproduce the ideal simulator job by job (Section 8.9). Every figure quoted in this paper is a development figure: the sealed parts of three traces, the main one's three terms among them, will be opened once after the method, the parameters and the primary metric have been frozen, and their numbers will be reported separately (Section 7.6). The empirical headline uses conservative history visibility, under which Guard(600) closes 0.429 of the FCFS-to-SJF gap at the busiest load, with interval $[0.357, 0.533]$, against 0.801 under the original result clock, which we retain as an optimistic reference (Section 8.3). And the guarantee is per job and additive, so a guarded run may still have a worse quantile than first-come first-served, by at most the promise (Section 9).

Section 2 places the paper in the literature and Section 3 states the model, the visibility protocol and the metrics. Sections 4 and 5 give the cost predictors and the scheduler, Section 6 the identity, the per-job bounds and their tightness, and Section 7 the traces. Section 8 reports the experiments and Section 9 the limitations; the deferred proofs and the tightness constructions are Supplementary Sections S4 and S8.

2. Related Work

2.1. Scheduling with predictions

Purohit, Svitkina and Kumar [22] frame predictions in online algorithms as a consistency–robustness trade-off. Predicted-size scheduling has since been studied through stochastic classes [23], bounded multiplicative error [11], worst-case distortion [24,25], many-server limits [26], prediction acquisition [27], and per-job stretch on a preemptive machine [13]; the survey of Mitzenmacher and Shahout [12] synthesises this line. Our reference is FCFS rather than an optimum, our scope is one job rather than a mean, arrivals and prediction errors are arbitrary, and service is non-preemptive. The resulting bound is correspondingly limited: it controls additional waiting relative to FCFS and does not assert an improvement. Its usefulness is empirical and depends jointly on size dispersion, predictability and arrival structure, not on membership in a necessary heavy-tail class.

2.2. Fairness relative to FCFS, and bounded overtaking

Worst-case per-job fairness predates fair-start-time metrics. Schwiegelshohn and Yahyapour [28] call a schedule λ -fair when jobs submitted after job i cannot increase i 's flow time by more than a factor λ , and bound fairness and efficiency together for a preemptive gang policy. Their bound is multiplicative and, for a short job, tighter than an L -denominated one; it is obtained by gang preemption. Our guarantee is instead additive,

non-preemptive, and referenced to the same input under FCFS. Proposition 3 says only that a wrapper of our family, protecting against every opaque base proposal sequence, cannot uniformly promise below order L once it permits overtaking; it is not an impossibility for all scheduling mechanisms. Supplementary Section S9 gives their constants and the rest of the per-paper detail this section compresses.

Fair-start-time work gives a closer reference: a job is compared with a re-simulation that excludes later arrivals [29], and the resulting excess is then used as a job-level metric [30,31]. Conservative backfilling instead fixes a reservation for every queued job, so later backfills cannot delay that promised start [32]; it is a $G = 0$ endpoint relative to the reservation, not relative to same-input FCFS. PV-EASY [33,34] enforces zero lower-priority delay by killing and restarting obstructing backfilled jobs. These guarantees are online but use different references or preemption, whereas ours limits a non-preemptive wrapper's additive FCFS excess.

The closest architectural precedent for a wrapper around an arbitrary policy is Load Balancing Guardrails [35], which augments any dispatching policy so that a load-balancing system with SRPT at each server is mean-response-time optimal in heavy traffic. Its reference, stochastic objective, preemptive server policy and dispatch-level constraints differ from our per-job additive promise, but the separation between an unrestricted base decision and a protective wrapper is shared. At one server our "pointwise maximal" statement uses the established supervisory-control meaning of a maximally permissive supervisor, disabling only the actions needed to keep behaviour within the legal specification [36]; we claim that property only for the opaque-base, integer-time class of Theorem 8, not as a general least-restrictive result. Supplementary Table S21 compares the reference discipline, arrivals, preemption, information, server model and guarantee across these nearest mechanisms.

Bounded overtaking has a history in queueing theory, in two currencies. Counting came first: Nudge lets a small arriving job swap past the job at the tail of the queue [37], Nudge- K lets it pass up to K others while no job is passed more than once [38], the line continues with Nudge*(M) [39], and the finite-skip models of WCFS restrict service to the oldest n jobs and add work conservation and non-idling [40]. Work is the second currency: γ -Boost measures the *crossing work* of later arrivals that precede a tagged job [14], and that line extends to k servers [15]. All of them are analysed in steady state, under Poisson arrivals with light-tailed or phase-type sizes, and target stationary means or tail constants rather than a deterministic per-job cap enforced on every sample path. That Nudge implies, on every sample path, a per-job additive bound of one small-job length is our own reading and not a claim of [37], whose analysis is stochastic by design, and we do not claim bounded overtaking at one server as new.

Within that conjunction, Theorem 7 supplies a matching achievable band on a specified batch family, and Theorem 8 supplies pointwise maximal permission in its opaque-base integer-time class. Neither claims global instance optimality or a least-restrictive wrapper for $k \geq 2$.

Our contribution is therefore a conjunction and not a new currency. The identity is two-sided and holds on an arbitrary arrival sequence with arbitrary service times, under no distributional assumption; the constant $(2 - 2/k)L$ is exact for k non-preemptive servers, as is the $(3 - 2/k)L$ of the wrapper; the converse makes a work budget necessary and not merely sufficient, up to an additive $O(kL)$; and Algorithm 1 enforces the bound online around any base policy, learned ones included, under predictions that may be arbitrarily wrong.

The additive shape and the constants are familiar on their own. Two-sided additive bounds against a fluid or ideal reference are the standard form in packet scheduling [41–

44], where the error term is one maximum-size packet precisely because that packet cannot be preempted [45]; the coefficient $2 - 1/k$ of Graham's list-scheduling bound has the shape of the $(2 - 2/k)$ and $(3 - 2/k)$ above, and the additive form appears inside his proof [46], while non-preemptive real-time feasibility conditions charge a whole uninterrupted execution for the same reason [47]. What we prove is not a conservation law in the sense of Kleinrock [48] and the achievable-region work after it, whose sample-path members are aggregate or class-level identities rather than per-job comparisons against a named reference [49,50]; the law does extend to several servers [51], but under non-preemptive priority with $k > 1$ it needs every class to share one service-time distribution [52], which is exactly what our model withholds. Lemma 1 is the quantitative substitute, and it differs from the nearest pathwise work-difference bound [53, Lemma 2] in comparing two arbitrary non-preemptive work-conserving k -server policies, in bounding total unfinished work, and in being two-sided. Aging disciplines [54,55], the unfairness analyses of SRPT against processor sharing [56,57] and the multiserver accumulating priority queue [58,59] describe classes in steady state and not individual jobs, whereas our $\text{over}[q]$ accumulates overtaken work and carries a pathwise cap. The word itself needs one caution: in queueing theory *overtaking* means departing before a customer who arrived earlier [60,61], while we measure the work that passes a waiting job. A single-server argument also stops working with several servers, the workload vector recursion being Kiefer and Wolfowitz's [62] and its minimality under first-come first-served Daley's [63], which holds only where arrivals are allocated to servers in a manner that does not depend on their service times; our identity replaces that step with a quantitative comparison. Supplementary Section S9 states each of these bounds in the form its source gives it.

2.3. Runtime prediction and scheduling in deployed systems

Backfilling schedulers on high-performance computing systems have used predicted runtimes in place of user estimates for two decades [64], with predictions learned from past jobs and queue reordering tuned online [65,66]; their resources are multi-dimensional and their reported metric is a mean bounded slowdown. Closest to our setting, Zhang, Wu and Lu [67] predict the running time of submitted code in a judging queue and dispatch in ascending predicted runtime inside fixed-size windows, so we are not the first to schedule a judging queue by predicted cost; a fixed window does limit overtaking, but by position, and that work reports no per-job bound and no adversarial-prediction test. In inference serving, where non-preemption is close to a hard constraint, recent systems learn a relative order instead of a length [68] or target the tail because prediction-driven policies are fragile under distribution shift [69,70], and production systems otherwise avoid starvation with aging counters and maximum waiting times, without proofs. Serverless platforms are in the same position, SFS approximating shortest-remaining-time-first from user space [71] and ALPS learning a per-function rank and time-slice cap from an offline replay [72], neither with a bound. Our guard is a wrapper rather than a policy, so it can be placed around any of these schedulers, and its bound does not depend on which one is underneath.

2.4. Time-varying and deadline-driven load

That people concentrate their actions near a cutoff has been measured directly, in time management research [4] and in conference registration data [5,6]. The queueing consequences of an arrival rate that varies on the timescale of the service time are also well studied: peak congestion in a multi-server system lags the peak of the arrival rate [73], and staffing rules that follow a time-varying rate rest on offered-load approximations rather than stationary formulas [74–76]. These results say how many servers to provide. They give no waiting bound for an individual job and no guidance for a queue that has

already formed, and we leave capacity fixed. The steady-state classified-priority formula that motivates ranking by predicted cost [23,77] does not survive contact with this load either; Section 3 measures the mismatch, which is why we state the guarantee on sample paths and assume nothing about arrivals.

2.5. Relational and tabular models for cost prediction

Our comparison includes a heterogeneous graph neural network over users, tasks and assignments, in the relational graph convolution family [10,78]. Two features of the published benchmarks make them a poor argument either way. In RelBench [79] and its successor [80] the gradient-boosted baseline is fitted on the raw columns of the entity table alone, without hand-built aggregates over related tables, so the margin reported there is not a comparison against a tabular model carrying engineered history features; and the same paper's study against a human data scientist, which is the closer comparison, records regression as the region where the relational model struggles. Cost prediction is a regression task on entities with long individual histories, which places it in that region, and the evidence that tree ensembles remain strong on tabular data concerns single tables [81]. We therefore settle the question inside our own protocol instead of importing a conclusion, with a tabular baseline that carries engineered causal history features [9], an ablation ladder that removes and then permutes the edges, and a comparison made on queueing outcomes rather than on AUROC alone. The negative result we report is about this task on these traces, where each user and each task already has enough history of its own; the only relational signal we find is on task types that have none yet.

3. Problem Model

3.1. Jobs, Servers and Dispatch

A finite set of n jobs is served by k identical servers. We use *server* throughout for what a deployment would call an executor, a judge host or a worker, and *job* for what a platform would call a submission, an invocation or a run. Job i arrives at time $a_i \geq 0$ and requires C_i units of service. Service is non-preemptive, so a server that starts a job keeps it until the job finishes or until it has been running for the per-job time limit L , at which point the system stops it. Throughout, C_i denotes executed work, so $0 < C_i \leq L$ by construction and a job stopped at the limit contributes exactly L . An enforced timeout terminates such a job without a successful result; the waiting-time guarantee does not bound that loss of completion quality. Observed maxima and analysis truncations in a trace are distinguished from enforced timeouts in Section 7. One quantity carries three names in this paper: we write *cost* in the application sections, *service* or *service time* in the analysis, and *work* when several jobs are summed, and all three denote C_i . Two further words are fixed here because the application sections need them. A *task* is the exercise or problem a job is an attempt at, so many jobs share one task; a *run* is the unit of work of a continuous-integration pool, and is a job in the sense above.

Servers are work-conserving, meaning that no server idles at an instant at which some job waits. Jobs carry a rank, the total order induced by the pair (arrival time, input index); we write $j < i$ for " j has the smaller rank", which implies $a_j \leq a_i$. Events occurring at a common instant are processed as completions, then arrivals, then dispatches. A policy is any rule that, whenever a server is free and at least one job waits, names one waiting job and dispatches it; it may be randomised, learned, priority-driven, or fed by predictions of arbitrary quality, and we assume nothing else about it.

Arrival times are arbitrary. We place no stochastic assumption on a_i , on C_i , or on the dependence between them, because the load we are concerned with is generated by an external calendar and is neither stationary nor Poisson.

3.2. What the Scheduler Knows

The service time C_i is hidden at arrival. The scheduler reads it at the instant i completes and not before. Any service that runs submitted work without inspecting it first is in that position. Available at a_i are a feature vector x_i and a score $\hat{C}_i$ computed from it.

Visibility is causal, and we define it by result availability instead of timestamp order. Write done_j for the instant at which the outcome of job j became readable and $\delta \geq 0$ for the delay with which outcomes reach the feature store. Then

$$\text{information about } j \text{ may enter } x_i \iff \text{done}_j + \delta \leq a_i.$$

Historical features are recomputed under the chosen release rule rather than assembled once and masked afterwards. The original prediction study in Section 4 uses the source platform's result clock. In a replayed queue, an endogenous result instead becomes available at $\text{done}_j^P = a_j + W_P[j] + C_j$. Source-clock causality therefore does not imply availability under policy P : a recorded result may already be visible while its replayed job is still waiting. The conservative headline restricts history to replayed submissions in the same class-term copy and delays its source release; the static control uses no outcome history. Section 8.3 states their availability checks, separates external history from same-copy queueing exposure, and describes the pending policy-specific refinement. The guard theorem is independent of which scores these protocols produce.

3.3. Waiting Time, the FCFS Reference and the Service Guarantee

Let s_i^P be the start time of job i under policy P and $W_P[i] = s_i^P - a_i$ its waiting time, which excludes service. FCFS is the policy that always dispatches the waiting job of smallest rank, and that dispatches in increasing rank when several servers become free at the same instant. We run it on the same arrival sequence, with the same service times and the same k , so that the comparison is pathwise rather than distributional. The per-job excess is

$$\text{excess}_P[i] = W_P[i] - W_{\text{FCFS}}[i].$$

Two further quantities carry the analysis. Writing $j \prec i$ for " j is dispatched before i " in the dispatch sequence,

$$\text{In}_i = \sum_{j \succ i, j \prec i} C_j, \quad \text{Out}_i = \sum_{j \prec i, i \prec j} C_j,$$

the work that overtook i and the work that i overtook. Section 6 relates them to $\text{excess}_P[i]$.

We say that a policy P offers a service guarantee G if $\text{excess}_P[i] \leq G$ for every job i , on every arrival sequence, for every vector of service times bounded by L , and for every score, adversarial scores included. Two words are kept apart from here on: the *promise* is the number G the operator states, and the *guarantee* is the proved statement that $\text{excess}_P[i] \leq G$ holds for every job. An operator chooses a promise; a theorem supplies a guarantee. The statement is per job and holds on the realised sample path, in no sense in distribution or on average, and it does not presume that the score is any good. A per-individual additive bound relative to a reference discipline is familiar from packet scheduling [42], and the same difference has been measured after the fact as a fairness diagnostic in batch schedulers [29,30], where the fair start time is defined in two variants; the reference used here, first-come first-served with no backfilling, is the relaxed one. The robustness results in the scheduling-with-predictions literature are of a different kind, comparing against an optimal policy in an average sense under a bound on the prediction error [11,22].

We take FCFS as the reference because it is what these services deploy by default, so the excess is a quantity an operator can be held to. It also has known extremal properties for the unfinished-work vector in a multiserver system: the FCFS workload vector is minimal in the weak-majorization order among systems whose allocation of arrivals to servers does not depend on service times [63], a hypothesis narrower than work conservation alone. That extremality is why single-server arguments do not transfer unchanged to $k > 1$.

3.4. Metrics

The primary metric is the 99th percentile of waiting time inside deadline windows, and we write $p99_{dl}$ for it, the symbol the tables carry as a column head. A deadline window is the 24 hours preceding a deadline; a job belongs to the window if and only if its arrival falls inside it, and a job that arrives inside but completes after the window closes is still counted. We fix this choice before any sealed part of a trace is opened. We also report four secondary quantities. The mean wait over all jobs. The maximum of $excess_p[i]$, the quantity a service guarantee bounds. The *harm*, the largest excess over FCFS among the jobs FCFS would have started within 1 s, that is, among the jobs FCFS would not have delayed at all; a guarantee G says nothing about how that excess is distributed, and the harm is the quantity an operator's users complain about. The *firing rate*, the share of dispatch epochs at which the guard of Section 5 overrides the base policy, weighted by queue length. We also report the worst wait among heavy jobs, heavy being the label Section 4 defines.

Improvements are also expressed as the fraction of the gap closed,

$$\frac{\overline{W}_{FCFS} - \overline{W}_P}{\overline{W}_{FCFS} - \overline{W}_{SJF}},$$

where SJF sorts waiting jobs by their true service times. SJF on true sizes is a reference policy and not an optimum. It is not deployable, since C_i is unknown at arrival, and it does not minimise any of the metrics above on a non-preemptive multiserver system. The ratio is unstable when its denominator is small, so we mark such configurations and print the two absolute waits next to every ratio. The fraction of the gap closed is not a percentage reduction in waiting time and we do not report it as one.

3.5. Steady-State Models and This Load

Stationary queueing models give closed forms for the mean wait of a non-preemptive priority class [77] and would let class thresholds be placed analytically. They need a stationary arrival process, which deadline-driven load does not supply. We measured the mismatch. On a synthetic Poisson stream whose service distribution matches the first two moments of the real one, the formula agrees with simulation to 0.4%. On the same workload replayed at its recorded timestamps, the measured wait is 10.7–13.7 times what the formula predicts on the development trace. A threshold tuned on the formula would be tuned against a number that is off by an order of magnitude.

We therefore use steady-state expressions as motivation only, and every promise made in this paper is a per-job statement on the realised sample path (Section 6). The mechanism that makes reordering worth doing is the three-job arithmetic of the introduction, where total work is unchanged while one long job moves back a little and several short ones move forward a lot.

4. Cost Prediction

The scheduler of Section 5 needs a number at the instant a job arrives. This section says what that number estimates, how we keep its inputs causally visible, which model families we compared under that one protocol, and which model we send into the queue. All figures are development figures, and the traces they come from are described in Section 7.

4.1. Two Targets

A scheduler and an evaluation want different things from the same model. The conditional mean $\mathbb{E}[C_i | x_i]$ is the single-server pairwise-interchange target and is the candidate score fitted in Section 5.1; that section also explains why the argument does not establish optimality on multiple servers, where we evaluate the order empirically. For comparing predictors we also need a target that stays interpretable when the mix of jobs shifts between periods, so we score each model on a binary label. A job is *heavy* when its executed cost C_i exceeds the 95th percentile of the training rows; this is the one definition of heavy used in the paper, and the worst heavy-job wait of Section 3 is taken over the jobs it selects. On the main judging trace that cut-off is 1.559 s and heavy jobs are 1.31% of the test period; on the second judging trace it is 1,314 ms and 5.07%. The label is deliberately fixed on training rows and never refit on the period being scored, so its rate in the test period is a quarter of its rate in training. Because the positive class is small we report average precision next to AUROC, and we report RMSE on $\log(1 + C)$ and Spearman correlation for the regression output, since a ranking policy consumes order rather than level.

4.2. The Visibility Protocol, Made Operational

Section 3 defines visibility by result availability: information about job j may enter x_i exactly when $\text{done}_j + \delta \leq a_i$. Turning that into a feature pipeline takes three commitments.

First, every historical feature is recomputed by streaming the log in availability order and reading each entity's state as of the query instant. No feature is assembled over the whole period and masked afterwards. We test this by recomputing the features from scratch and comparing: 0 rows disagree in any of the eight configurations. Supplementary Section S1 gives the audit in full, including the same test applied to the natural but wrong construction and the separate checks on the graph pipeline.

Second, thresholds are refit per target period. The heavy cut-off and the class boundaries used inside the models are recomputed from the training rows of that target, and each target period is predicted by a model fitted only on periods before it, frozen before the target's first arrival. Training rows whose results would not have been available by that instant are dropped; on the primary configuration the rule binds on 0 rows. We report the zero because a rule that never binds is easily mistaken for a rule that is not there.

Third, we treat the delay δ with which results reach the feature store as a sensitivity and not as an assumption; Supplementary Section S1 gives the two clocks in full, on which ranking survives a ten-minute feedback delay while the probability calibration degrades.

All families see the same rows, the same split and the same availability rule. Supplementary Section S1 defines them: the gradient-boosted tabular groups M1 (task history), M2 (user history), M3 (static features of the submitted artefact), M4 (everything tabular) and M5 (M4 plus one-hop relational aggregates), with the scrambled-neighbourhood control M6; the sequence family R1 and R1S, a GRU over the user's own earlier jobs used alone and as a state vector inside the boosted model; and the relational ladder N0, G0U, G0, G1 and G2, which runs a heterogeneous network from a multilayer perceptron on M4's columns through empty neighbour slots to the real edges and then to edges permuted inside a period and time block.

4.3. What the Comparison Shows

Supplementary Table S2 gives the two judging traces. One-hop relational aggregates add nothing on either: M5 – M4 is +0.0019 of AUROC on the first and –0.0002 on the second. The ladder tells the same story with a trained model in place of hand-built aggregates. Deleting every edge (G0) and permuting the edges (G2) leave AUROC within 0.002 of G1, and the paired intervals for G1 – G0 (–0.0001 [–0.0019, 0.0018]) and G1 – G2 (+0.0019 [–0.0000, 0.0042]) cover zero. The one step of the ladder that resolves is N0 → G0U, +0.0139 [+0.0090, +0.0195] of AUROC at identical information, so what moves there is the encoder, with the edge set held empty in both.

Edges do carry something, and it is confined to cold-start tasks. On the 1,092 test rows whose task has fewer than five visible prior results, G1 beats M4 by +0.1464 [+0.0997, +0.1887] of Spearman correlation, and beats its own permuted-edge control by +0.0282 [+0.0130, +0.0432]. Over all rows the G1 – G2 Spearman difference is +0.0063 [+0.0014, +0.0111], small and resolved; the corresponding AUROC and AUPRC differences are not. We read this as a task property rather than a verdict on relational models: when each entity's own recent record is visible and rich, a neighbourhood adds little, and the place where it adds anything is the entity that has no record yet.

The sequence state has the best prediction scores of anything we fitted, and no consistent effect in the queue. R1S reaches AUROC 0.9366 against M4's 0.9283, a difference of +0.0083 [+0.0038, +0.0131], and it also wins on RMSE and Spearman. Inside the queue it is not a consistent improvement: against the same ranking objective it gains +0.007 [0.003, 0.011] of the deadline-window gap at busy-hour utilisation 0.8, loses –0.010 [–0.022, –0.002] at 0.5, and is unresolved at 1.0. A control fitted on the same 90% of rows that the GRU's early stopping left available moves the same way, so the smaller training set does not explain it.

Which information matters is not a constant across platforms. On the first judging trace the user's own history and the static features of the artefact beat the task's history; on the second the task's history beats the user's by 0.11 of AUROC (Supplementary Table S2). A task on the first platform is attempted by one cohort within a few days, so who is writing still carries information; a task on the second is attempted by thousands of users over years, so the task's own record has already fixed the cost. The non-judging traces push this further. On the serverless trace the identity of the function explains 0.867 of the variance of $\log(1 + C)$ on its own, and static and calendar features alone are close to useless for ranking (Spearman 0.021). The two CI pools sit at opposite ends, entity history giving AUROC 0.9968 on one and 0.6334 on the other, because that pool's job labels do not separate its costs; and the compute-farm trace is predictable in rank and not in level. Supplementary Table S1 gives all four traces by feature group; the AUROC column of Table 7 carries the figure the scheduling results are read against.

4.4. Cost of Predicting, and What Enters the Scheduler

Prediction has to be cheap enough to sit in front of a queue whose median job is 204 ms, and it is: Supplementary Section S1 gives the measured per-job cost of each family, and charging it to the predictive policies before they enqueue changes the closed gap by less than 0.001 at every load we ran.

It is a queueing decision. M5 has the higher AUROC of the two tabular candidates and loses in the queue at all three loads, by 0.012, 0.008 and 0.010 of the deadline-window gap, with intervals excluding zero in each case. R1S has the higher AUROC of everything we fitted and does not win consistently. We therefore take M4, the full tabular model, into Section 5: it was selected on the queueing outcome on validation periods, not on prediction quality, and the two orderings do not agree.

5. Scheduling Algorithm

The scheduler has two layers. A base policy orders the queue by a score computed at arrival, and a wrapper limits how much work may overtake any waiting job. The two are independent, in that the wrapper accepts any base policy and its guarantee does not depend on that policy behaving well.

5.1. Ranking by Expected Cost

When a server frees, the base policy takes the waiting job with the smallest score. For a batch on one server, the pairwise-interchange argument orders jobs by their conditional mean service times. The same conclusion does not hold in general on $k \geq 2$ servers. For example, on two servers let $A = 2$, $C = 3$, and let B be 0.1 with probability 0.9 and 100 with probability 0.1. Ordering the three conditional means starts A and C and has expected mean wait $2/3$; starting A and B instead has expected mean wait 0.096667. Thus expected-cost ordering is an empirically motivated base policy here, not a multi-server optimality claim.

Both raw-scale squared error and the Tweedie mean deviance have the conditional mean $\mathbb{E}[C_i | x_i]$ as their unrestricted population optimum. Their fitted orders can nevertheless differ with finite model capacity, the link function, regularisation, sample weighting and optimisation. We therefore compare objectives empirically while keeping the features, model size and rolling-origin protocol of Section 4 fixed. SPJF-E uses raw seconds with a Tweedie loss of power 1.5, which accommodates mass near zero and a long right tail [82,83]; SPJF-log uses the same pipeline fitted by squared error on $\log(1 + C)$ and then inverted.

The literal inverse $\exp(\mathbb{E}[\log(1 + C) | x]) - 1$ is no larger than the conditional arithmetic mean, with equality only in the degenerate case. The gap is not determined by the conditional variance alone. A residual-distribution correction, such as the smearing estimator of Duan [84], is required to use a log-scale regression as a mean estimator under its stated conditions.

Only the order of the fitted score is used. Its level does not transfer across periods, since the share of heavy jobs in the training periods differs from the target period by a factor of several, so we never read the score as a quantity, neither as an estimated finish time nor as a threshold. The scheduler is therefore invariant to a monotone recalibration of the score.

Section 8.4 measures what the fitted objective is worth, together with the alternatives we tried and rejected: a predicted quantile, a two-class rule, and squared error on raw seconds. Its partial-oracle repair replaces selected scores with truth and is not invariant to an arbitrary monotone rescaling of the original output. It therefore diagnoses the errors of this fitted predictor; it does not establish a universal asymmetry of prediction errors or a different population estimand.

No choice of ranking function bounds the worst case. A job the score misjudges can still be passed indefinitely, whatever the score is and however it was fitted; Section 5.2 is what bounds it.

5.2. The Overtake-Budget Guard

Fix a base policy A . For every waiting job q and every time t define

$$\text{over}[q](t) = \sum \{ C_j : j > q \text{ in rank, } j \text{ has completed by } t \},$$

the total true service of the higher-ranked jobs that have finished by t . Such a job arrived at or after a_q , so it necessarily ran while q waited. Job q is allowed a budget $\text{budget}(q, t)$ on that quantity, and the fired set at a dispatch epoch is $E(t) = \{ q \text{ waiting} : \text{over}[q](t) \geq$

$\text{budget}(q, t) \}$. At every dispatch epoch, that is, at every instant at which a server is free and the queue is non-empty, the wrapper serves the minimum-rank member of $E(t)$ when $E(t)$ is non-empty and otherwise serves whatever A chooses (Figure 2).

Serving the minimum-rank member matters when budgets differ between jobs. It guarantees that whenever a job j is dispatched ahead of a lower-ranked waiting job i , the job i was not in $E(t)$, hence $\text{over}[i](t) < \text{budget}(i, t)$ at that very instant. The per-job bound of Section 6 rests on that step. With a constant budget $\text{budget}(q, t) \equiv B$ the quantity $\text{over}[q](t)$ is non-increasing in rank, so $E(t)$ is non-empty exactly when the queue head belongs to it and the rule degenerates to a test on the head. Setting $B = 0$ recovers FCFS.

The Cap Carries the Promise; the Shape Does Not.

The guarantee of Section 6 reads the budget only through an upper bound on it: if $0 \leq \text{budget}(q, t) \leq B_{\max}$ for every waiting job and every instant, then every job waits at most $W_{\text{FCFS}} + B_{\max}/k + (3 - 2/k)L$, for any base policy, any arrival sequence and any predictions. Everything below the cap is a design freedom that changes how much of the base policy's benefit survives and which jobs pay for it, and changes the promise not at all. We study the one-line family

$$\text{budget}(q, t) = \min(B_0 + \gamma n_q + \eta k(t - a_q), B_{\max}),$$

with $B_0, \gamma \geq 0$, $0 \leq \eta < 1$ and n_q the number of jobs waiting when q arrives, and read three shapes off it: *constant* ($\gamma = \eta = 0$), *age-relative* ($\eta > 0$), and *queue-length* ($\gamma > 0$). The coefficient η says that a waiting job tolerates at most a fraction η of the system's entire service capacity, over its own waiting time, being spent on jobs that arrived after it, so the allowance grows with the wait a job has already had; being a fraction of capacity it is dimensionless and needs no retuning when k or the load changes. The term γn_q instead widens the allowance only for jobs that arrive into a queue that is already long, and is flat in the wait. B_0 , γ and $B_{\max}$ are measured in work and scale with k .

Only the age-relative shape carries a second, multiplicative bound, $W < (W_{\text{FCFS}} + B_0/k + (3 - 2/k)L)/(1 - \eta)$, and only when $\gamma = 0$: with $\gamma > 0$ the constant part of the budget differs from job to job, so the multiplicative form holds per job with that job's own constant, and no single number is available for the operator to state in advance (Supplementary Section S4.6). For $\gamma > 0$ we claim the additive bound alone.

Parameters from a service promise.

The operator states G , the number of seconds by which any job may exceed its FCFS waiting time, and reads off $B_{\max} = k(G - (3 - 2/k)L)$, the inversion of the uniform certificate proved in Section 6. No data are needed for this step. Corollary 3 issues no positive budget below $(3 - 2/k)L$; the zero budget remains admissible and reproduces FCFS exactly. Proposition 3 separately gives an L -scale floor only for this wrapper family when it must protect against every opaque base-policy proposal sequence. The promise G follows from $B_{\max}$ alone. The shape parameters B_0 , γ and η decide how much of the base policy's benefit survives and which jobs pay for it, so we choose them on a validation trace that excludes the test period; that pool is held out of the selection and not of the development process, and Section 8.2 states what the selection therefore does and does not establish. The rule is fixed before the grids are run, and Supplementary Section S5 states it in full together with what each grid selected. Supplementary Table S4 gives the grids and what they selected.

Algorithm 1 Overtake-budget guard wrapped around a base policy A

```

1: parameters: cap  $B_{\max} \geq 0$ ; shape  $B_0, \gamma \geq 0, \eta \in [0, 1)$ ; base policy  $A$ 
2: state: waiting set  $Q \leftarrow \emptyset$ ;  $\text{over}[q] \leftarrow 0$  for every job  $q$ 
3: while some job has not started do
4:    $t \leftarrow$  next event time
5:   for all jobs  $c$  completing at  $t$  do  $\triangleright C_c$  becomes known here and nowhere earlier
6:     for all  $q \in Q$  with  $q < c$  in rank do
7:        $\text{over}[q] \leftarrow \text{over}[q] + C_c$ 
8:     end for
9:   end for
10:  for all jobs  $i$  arriving at  $t$  do
11:     $n_i \leftarrow |Q|$ ;  $Q \leftarrow Q \cup \{i\}$ ;  $\text{over}[i] \leftarrow 0$   $\triangleright n_i$  fixes  $i$ 's constant part
12:  end for
13:  while a server is free and  $Q \neq \emptyset$  do  $\triangleright$  dispatch epoch
14:     $E \leftarrow \{q \in Q : \text{over}[q] \geq \min(B_0 + \gamma n_q + \eta k(t - a_q), B_{\max})\}$ 
15:    if  $E \neq \emptyset$  then
16:       $j \leftarrow$  the minimum-rank member of  $E$   $\triangleright$  guard fires
17:    else
18:       $j \leftarrow A(Q, t)$   $\triangleright$  base policy decides
19:    end if
20:    dispatch  $j$  on the free server;  $W[j] \leftarrow t - a_j$ ;  $Q \leftarrow Q \setminus \{j\}$ 
21:  end while
22: end while

```

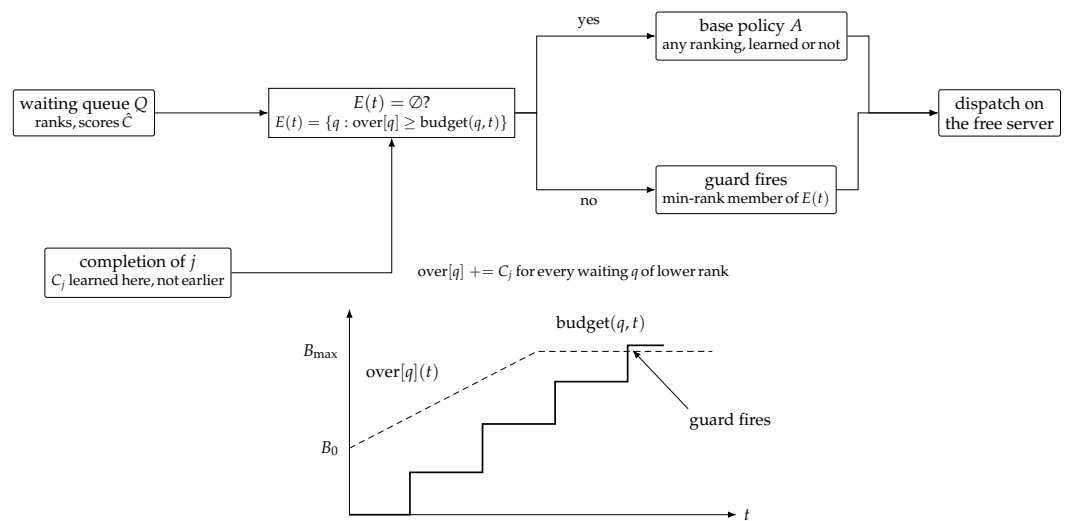


Figure 2. The guard of Algorithm 1 wrapped around a base policy (above) and the accounting it runs for one waiting job (below). Only completed work is charged, because C_j is unknown until j finishes, so $\text{over}[q]$ is a step function with a step at each completion of a higher-ranked job; at a dispatch decision, q is eligible if its account reaches its current budget. The guard dispatches the smallest-ranked eligible job, which need not be q . The budget drawn is the age-relative shape, rising at rate ηk from B_0 and capped at $B_{\max}$; the guarantee depends on the cap alone, so any curve under the dashed ceiling carries the same promise. The lower panel is schematic. Sources: the definitions in Section 5.2; $B_{\max} = k(G - (3 - 2/k)L)$ is Corollary 3.

What is charged, and why no service time is needed at dispatch.

The budget is charged the true executed work of overtakers, and only when they complete. At the instant the guard decides, it has read C_j only for jobs that have already finished, and a job still in service contributes nothing. That is what makes the wrapper implementable in a system that learns a job's cost at its completion, and it is also why the bound carries an additive term in place of an exact accounting. When the last overtaker of i is dispatched, up to $k - 1$ further overtakers may be in service with their work not yet charged, and the overtaker being dispatched is not charged either, so the work that actually passes i can exceed the budget by at most kL . Under the timeout at L , executed work means $\min(C_j, L)$; the log records that quantity and the guard charges it, so no assumption on raw sizes enters.

Cost per event.

Algorithm 1 is written for readability rather than for execution, since its inner loop over waiting jobs at each completion is linear. A Fenwick tree over completed work and a segment tree over the window of waiting ranks make each event cost $O(\log n)$, with all budget arithmetic carried in integer microseconds so that the firing test is exact and no floating-point clock enters the schedule. Supplementary Section S5 gives the two structures and the rewriting of the firing test.

Why the budget is measured in work and not in positions.

Earlier bounded-overtaking mechanisms limit how many times a job may be passed, which is the currency of the finite-skip and Nudge- K rules [37,38,40]. Counting is not necessary, and it is sufficient in this model only through the worst-case charge of L per overtake, so a count of κ buys a promise of $(\kappa + 2k - 2)L/k$ (Remark 1, (2)) and is loose by the ratio $L/\bar{C}$, one to two orders of magnitude on our traces. Supplementary Section S4.9 gives the argument in full, and Section 8 runs the count rule as a baseline and measures how much the looseness costs on a real load.

5.3. Compared Policies

Table 1 names the policies evaluated in Section 8 and fixes the names used in every table of this paper. Guard(G) wrapped around SPJF-E is the method; the others separate its parts and attack it.

Every simulation that includes the guard asserts the per-job bound of Section 6 job by job inside the run (Section 8.6).

6. Theory

Everything below is pathwise: one arrival sequence, one vector of service times, one realisation of whatever randomness the base policy uses. No stationarity, no independence and no assumption about score quality enters any proof, and the upper bounds hold against a base policy that knows the future. An identity ties a job's delay relative to first-come first-served to the work that overtook it; two bounds turn that identity into the guarantee of Algorithm 1; and a set of constructions shows that each constant in those bounds is an exact supremum. Every statement is given here with the idea of its argument; Supplementary Sections S4 and S8 give every deferred proof and every extremal construction, S8 opening with the idea of each construction beside it.

6.1. Conventions

We use the model of Section 3 throughout, and fix three readings the proofs depend on. The dispatches at one instant form a *phase*, ordered as the policy performs them, and

Table 1. Policies compared in the experiments, and the name each carries throughout.

Name	Role
FCFS	Deployed baseline, and the reference against which the guarantee is stated
SJF	Sorts waiting jobs by their true service times. Undeployable reference policy, not an optimum; the denominator of the fraction of the gap closed
SPJF-E	Ranks by the expected-cost score of Section 5.1, unguarded. Transmission from prediction quality to waiting time
SPJF-log	The same pipeline fitted on the log scale, and the other candidate scores of Section 5.1, run through the same simulator
Guard(G)	SPJF-E wrapped in the overtake-budget guard at the promise G , with the budget shape the selection rule of Supplementary Table S4 picks over all three grids together. This is the method
Guard-fixed(G), Guard-age(G), Guard-queue(G)	The same wrapper and the same promise, with the shape the rule picks inside the constant, age-relative and queue-length grid respectively. Ablation rows: they isolate what the shape is worth at a fixed cap, and one of them coincides with Guard(G) at each promise
Fixed(G)	The same wrapper at the equal-promise budget $\text{budget} \equiv B_{\max}$. It carries the promise G but not the harm constraint, and the selection rule rejects it as infeasible; it is reported as the comparator that spends the whole cap
Skip(G)	Position-count guard, finite-skip family, counted at dispatch with $\kappa = \lfloor Gk/L \rfloor - (2k - 2)$ from (2), the largest count that still promises G under that accounting
Corrupted scores	SPJF-E and Guard(G) under a reversed order, a random order, and a score that calls the truly longest 1% the shortest; tests the guard where the base policy is worst
Counterexamples	Retained traces from two audits and adversarial review; every job must satisfy the bounds of Section 6

the dispatch sequence behind $j \prec i$ is the concatenation of the phases; first-come first-served dispatches in increasing rank inside a phase. A job dispatched in the same phase as i executes no work during $[a_i, s_i^P)$, that interval ending at the instant of the phase, yet contributes its whole C_j to In_i . Remaining work at a_i is read *after* the arrivals at a_i , so a job of rank below i arriving at exactly a_i counts with its full service time.

6.2. The workload gap between two policies

Write $U_Q(t)$ for the unfinished work at time t under policy Q , counting every job with $a_j \leq t$, and $N_Q(t)$ for the number of jobs present. The unfinished-work vector of a k -server queue obeys the Kiefer–Wolfowitz recursion [62], and first-come first-served minimises that vector in the weak-majorization order for systems in which arrivals are allocated to servers in a manner that does not depend on their service times [63]. The policies compared below are not restricted that way, so what carries the single-server argument to $k > 1$ here is a two-sided bound between two arbitrary policies rather than an ordering.

Lemma 1 (Workload comparison). *Let Q_1 and Q_2 be non-preemptive work-conserving k -server policies run on the same input. Then $|U_{Q_1}(t) - U_{Q_2}(t)| \leq (k - 1)L$ for every t . For $k \geq 2$ the inequality is strict, $|U_{Q_1}(t) - U_{Q_2}(t)| < (k - 1)L$, and the constant is the exact supremum, approached but never attained. At $k = 1$ both sides vanish: one server executes work at rate 1 whenever any job is present, so every work-conserving policy has the same unfinished work at every instant.*

The difference $D = U_{Q_1} - U_{Q_2}$ is continuous and piecewise linear, and it cannot increase at any instant at which U_{Q_1} exceeds $(k - 1)L$, since more than $k - 1$ jobs are then present under Q_1 and every server of Q_1 is busy; tracking D back to the last instant at

which U_{Q_1} was below the threshold gives the bound. The proof, the strictness, and the cascade that shows the constant cannot be lowered are in Supplementary Sections S4.1 and S8.1.

6.3. The net-overtake identity

For a job i and a policy Q , let R_i^Q be the remaining work at time a_i , under Q , of the jobs of rank below i , read after the arrivals at a_i ; and let ρ_i^Q be the remaining work at s_i^Q of the jobs dispatched strictly before i in Q 's dispatch sequence that are still in service then. Write

$$\Delta_i = k(W_P[i] - W_{\text{FCFS}}[i]) - (\text{In}_i - \text{Out}_i).$$

Theorem 1 (Net-overtake identity). *For every non-preemptive work-conserving k -server policy P , every input and every job i ,*

$$\Delta_i = (R_i^P - R_i^{\text{FCFS}}) - \rho_i^P + \rho_i^{\text{FCFS}}, \quad |\Delta_i| \leq 2(k-1)L,$$

with strict inequality for $k \geq 2$; equivalently $|W_P[i] - W_{\text{FCFS}}[i] - (\text{In}_i - \text{Out}_i)/k| \leq (2 - 2/k)L$. At $k = 1$ the error term vanishes and $W_P[i] - W_{\text{FCFS}}[i] = \text{In}_i - \text{Out}_i$ exactly.

Job i waits throughout $[a_i, s_i^P]$, so every server is busy there and exactly $k W_P[i]$ units of work are executed, none of it on i . Splitting that work by rank gives the busy-period accounting

$$k W_P[i] = R_i^P + \text{In}_i - \text{Out}_i - \rho_i^P, \quad \rho_i^P := \rho_i^{\text{old}} + \rho_i^{\text{new}}, \quad (1)$$

where ρ_i^{old} and ρ_i^{new} are the work still unfinished at s_i^P of, respectively, the jobs dispatched strictly before i and the overtakers, and a same-phase overtaker contributes C_j to both In_i and ρ_i^{new} and nothing to the executed work. The jobs counted in ρ_i^P occupy distinct servers and none of them occupies the server that takes i , so $0 \leq \rho_i^P \leq (k-1)L$. Under first-come first-served $\text{In}_i = \text{Out}_i = 0$, and Lemma 1 applied at a_i bounds $|R_i^P - R_i^{\text{FCFS}}|$ by $(k-1)L$, strictly for $k \geq 2$; subtracting the two instances of (1) gives the identity. Each direction uses only two of the three ranges, which is why the slack is $2(k-1)L$ and not $3(k-1)L$, and at $k = 1$ all three terms vanish. Supplementary Section S4.3 gives the proof in full.

Corollary 1 (A work budget is sufficient). *If a policy keeps $\text{In}_i - \text{Out}_i \leq Z$ for every job, in particular if it keeps $\text{In}_i \leq Z$, then $\text{excess}_P[i] \leq Z/k + (2 - 2/k)L$ for every job.*

6.4. The constant $2(k-1)L$ is exact

The error term of Theorem 1 is not an artefact of the proof, although neither search nor real input reaches it: the extremal instances are geometric cascades needing hundreds of jobs, which nothing in a replayed workload arranges. Section 8.6 and Supplementary Section S3.13 report how far below the constant the measured residual stays, on small instances and on the trace alike.

Theorem 2 (Tightness of Theorem 1). *For every fixed $k \geq 2$, service bound $L > 0$, and $\varepsilon > 0$ there are inputs, work-conserving policies P and jobs i with $\Delta_i > 2(k-1)L - \varepsilon$, and there are others with $\Delta_i < -2(k-1)L + \varepsilon$. The construction is first given at scale $\hat{L} = k^m$ and then every arrival time and service time is multiplied by $L/\hat{L}$; thus the limiting sequence keeps the stated L fixed. Hence $\sup |\Delta_i| = 2(k-1)L$, a supremum that is never attained.*

Both witnesses start from the cascade of Supplementary Section S8.1, which drives $U_P - U_{\text{FCFS}}$ to $(k-1)L(1 - ((k-1)/k)^m)$ in m rounds. The upward witness then injects $k-1$ long jobs, the victim, and a stream of unit jobs the base policy prefers, so that first-

come first-served starts the victim at once with $\rho_i^{\text{FCFS}} = (k - 1)L$ while P holds it until the stream runs out; the downward witness is the mirror image, with P dispatching $k - 1$ jobs of service L in the victim's own phase. Supplementary Section S8.2 gives both in full, with the exact values they attain. Non-attainment follows from Lemma 1.

The two directions do not have the same content: upward the extremal value is a statement about delay, while downward half of it is the bookkeeping convention that charges an overtaker dispatched in the victim's own phase, which executes nothing while the victim waits. Supplementary Section S4.3 gives the remark in full, with the same-phase shares, the executed-work variant under which the upward supremum is unchanged and the best downward witness falls to $(k - 1)L$, and the two reasons we keep the dispatch-sequence convention.

6.5. Necessity, and What a Count of Overtakes Costs

Theorem 1 has a converse. A policy that promises anything about individual jobs is forced to bound overtaken work.

Lemma 2. *Let i be a job, and suppose $\text{excess}_P[i] \leq G$ and $\text{excess}_P[j] \leq G$ for every job j counted in Out_i . Then $\text{Out}_i \leq k(G - \text{excess}_P[i] + L)$.*

Corollary 2 (A work budget is necessary). *If P satisfies $\text{excess}_P[j] \leq G$ for every job on some input, then $\text{In}_i \leq kG + (3k - 2)L$ for every job i .*

Lemma 2 holds because every job counted in Out_i starts inside a window of length $G - \text{excess}_P[i]$, and of the jobs one server starts inside such a window all but the last complete inside it; Corollary 2 then adds it to Theorem 1, where the terms in $\text{excess}_P[i]$ cancel. Supplementary Section S4.7 gives both proofs.

Together the two corollaries say that bounding In_i and offering a per-job guarantee are the same requirement, up to an additive $O(kL)$ in the budget. Neither says that Algorithm 1 is the least restrictive rule with its guarantee, and we do not claim it.

Remark 1 (Counting overtakes is not necessary, and is sufficient only at the worst-case charge). *Bounded-overtaking mechanisms in the literature limit how many times a job may be passed [37,38,40]. Inside the model of Section 3, where $0 < C_i \leq L$, a count bound does give a per-job guarantee; it is not necessary for one, and it buys the guarantee at a factor $L/\bar{C}$. Charged when an overtaker is dispatched, the reading a position counter in the prior art implements, a bound of κ passes gives $\text{In}_i \leq \kappa L$, and Corollary 1 turns that into*

$$W_P[i] \leq W_{\text{FCFS}}[i] + (\kappa + 2k - 2)L/k, \quad (2)$$

which is the accounting and the formula used wherever this baseline appears below; inverting it, a promise G admits $\kappa = \lfloor Gk/L \rfloor - (2k - 2)$ passes, the largest integer with $(\kappa + 2k - 2)L/k \leq G$. Either accounting charges every overtaker the worst case L , so the rule is loose against a work budget by the ratio $L/\bar{C}$, one to two orders of magnitude on our traces. Supplementary Section S4.4 proves both halves, with the instance outside the model on which a single overtake makes the excess arbitrarily large, the instance inside it on which the count is unbounded while every job stays inside its promise, why at most κ overtakers can be dispatched while i waits, and what charging the count at completion instead costs.

6.6. The guarantee of the guard

We now bound what Algorithm 1 does. Nothing below restricts the base policy, which may be randomised, learned, or clairvoyant.

Theorem 3 (Guard bound). *Fix any base policy A and any cap $B_{\max} \geq 0$, and run Algorithm 1 with any budget rule that respects the cap, that is, with any function $\text{budget}(q, t)$ satisfying $0 \leq \text{budget}(q, t) \leq B_{\max}$ for every waiting job q and every dispatch epoch t ; the rule may depend on the job, may decrease over time, and may take different values at two dispatches made at the same instant. For every input and every job i , writing $W = W_P[i]$ for the wait under the wrapper,*

$$\text{In}_i < B_{\max} + kL, \quad (3)$$

$$W < W_{\text{FCFS}}[i] + B_{\max}/k + (3 - 2/k)L, \quad (4)$$

where (3) is asserted whenever i has at least one overtaker.

The proof reads the last overtaker of i : at the instant it is dispatched, i is waiting and is not in the fired set, so $\text{over}[i]$ is below its budget there; the overtakers dispatched earlier and not yet finished occupy at most $k - 1$ distinct servers, and the last one adds a further L , which is the kL of (3). Theorem 1 with $\text{Out}_i \geq 0$ turns that into (4). Supplementary Section S4.5 gives the proof in full.

Nothing in the proof reads the budget rule except through the cap, so the shape of the budget below the cap is a free choice and the promise is unaffected by it. One shape carries a second bound, in which the allowance grows with the wait the job has already had rather than being flat in it.

Theorem 4 (Multiplicative bound for the age-relative shape). *Suppose in addition that $\text{budget}(q, t) = \min(B_0 + \eta k(t - a_q), B_{\max})$ with one constant $B_0 \geq 0$ for every job and $\eta \in [0, 1)$. Then for every input and every job i ,*

$$\text{In}_i < B_0 + \eta kW + kL, \quad (5)$$

$$(1 - \eta)W < W_{\text{FCFS}}[i] + B_0/k + (3 - 2/k)L, \quad (6)$$

with (5) asserted whenever i has at least one overtaker.

The same computation runs at $t \leq s_i^P = a_i + W$, which bounds $\text{budget}(i, t)$ by $B_0 + \eta kW$; Supplementary Section S4.5 gives it in full. It goes through for a quantity fixed at i 's own arrival, so with the queue-length shape of Section 5, where $B_{0,q} = B_0 + \gamma n_q$ varies from job to job, the multiplicative bound stated with a common B_0 is false by a margin that grows with γ : Supplementary Section S4.6 gives the counterexample. For $\gamma > 0$ we therefore claim Theorem 3 and not Theorem 4. The cap, and with it the promise, is unaffected either way.

A scheduler that learns C_j at completion cannot charge work in progress, so only completed work is charged; the kL in (3) is the price of that and is what forces the additive term. Inverting (4) gives the parameter rule of Section 5:

Corollary 3. $B_{\max} = k(G - (3 - 2/k)L)$ makes Algorithm 1 offer the service guarantee G , whatever the budget shape below the cap and whatever the base policy A . The choice is available when $G \geq (3 - 2/k)L$, and gives a positive budget only when the inequality is strict; at equality $B_{\max} = 0$ and the wrapper is first-come first-served.

One assumption can be traded for a smaller constant. If the service enforces a per-job run-time cap $\ell_j \leq L$ known at arrival, charging it at dispatch and refunding it at completion keeps In_i within the budget, so the kL of (3) goes and the additive constant falls from $(3 - 2/k)L$ to $(2 - 2/k)L$, a drop of exactly L , the promise at $k = 1$ becoming excess $\leq B$ exactly. The information model is then not the completion-only one of Section 5

and the mechanism is conservative backfilling's [32,85], so Supplementary Section S7.6 carries it, with the single-server maximality and the counterexample that rules maximality out for $k \geq 2$.

6.7. Both constants of the guarantee are exact

Theorem 5 (Tightness of Theorem 3). *Take the constant budget $\text{budget}(q, t) \equiv B = B_{\max}$, and write $c(k) = \sup(k \text{ excess}_P[i] - B)/L$ over all inputs, base policies and budgets; by Theorem 3 the supremum over all budget rules that respect the cap B is the same number. Then*

1. *the term B/k is attained: on an explicit family the excess is exactly $\lceil B/k \rceil$, hence B/k whenever k divides B ;*
2. *every finite instance satisfies $(k \text{ excess}_P[i] - B)/L < 3k - 2$, while its supremum is exactly $3k - 2$ for every $k \geq 1$. Equivalently, the additive constant $(3 - 2/k)L$ of (4) cannot be lowered for any k .*

Part (i) is a queue of k jobs of service L , the victim, and a stream of unit jobs: the guard fires at $L + \lceil B/k \rceil$. For part (ii) the upper bound is Theorem 3, strictly; the lower bound runs the cascade for m rounds and then injects a sequence timed so that over $[i]$ stays below the budget $B = f + L + 1$ at three successive dispatch epochs — at 0, at f , and at $f + L = B - 1$, the last of them by a single unit — letting the base policy take every free server three times before the guard fires. Supplementary Section S8.3 gives both constructions and the values they reach.

Both sources of slack in Theorem 3, the constant of Theorem 1 and the kL of (3), are saturated on the same instance, and there the excess is real waiting time rather than bookkeeping, every overtaker completing before the victim starts. The additive constant $(3 - 2/k)L$ grows with k , from L at $k = 1$ towards $3L$, and search does not see this; Supplementary Section S8.3 reports what a hill-climb over small instances suggests instead, beside the size the witness actually needs.

6.8. When the Wrapper Changes Nothing

Proposition 1 (Consistency, one direction). *Run the base policy A alone. If $\text{over}[q](t) < \text{budget}(q, t)$ for every waiting q at every dispatch epoch of that run, then the wrapped run is identical to it, with the same dispatch sequence and the same waits. Consequently, for a constant budget B , each of the following implies $\text{guard}(A, B) = A$: $\max_i \text{In}_i^A < B$; or A itself satisfies $\text{excess}_A[i] \leq G$ on this input and $B > kG + (3k - 2)L$.*

The proof inducts on dispatch epochs and is in Supplementary Section S4.2, with the two counterexamples that show the converse fails and that there is no multiplicative consistency: the guard can fire and happen to choose what the base would have chosen, and the ratio of mean waits between the wrapper at $B = 0$ and a base ordering by service time is unbounded in L .

6.9. Which assumptions can be dropped

Proposition 2 (Relaxations). *In the setting of Theorem 1 and Theorem 3:*

1. *No global bound on service times is needed, provided the prefix maximum $\Lambda(t) = \max\{C_j : a_j \leq t\}$ is placed correctly, and the right place is not the same for every statement. The upward direction of Theorem 1, and hence Corollary 1, hold with L replaced by $\Lambda(a_i)$, since every job counted in R_i^P , R_i^{FCFS} or ρ_i^{FCFS} has rank below i and so arrived by a_i . The wrapper bounds (3) and (4), and likewise (5) and (6), need $\Lambda(s_i^P)$: the kL term of (3) charges the last overtaker and the at most $k - 1$ overtakers still in service beside it, all of them jobs of rank above i , which arrive after a_i and are only bounded by the prefix maximum at the*

- instant i starts. With $\Lambda(a_i)$ both are false, and by an unbounded margin: at $k = 1$ with budget $B = 2$, let i and one job of service 1 arrive at $t = 0$ and a job of service M at $t = 1$, with a base policy that takes the two others first. Then $\Lambda(a_i) = 1$ for every M , the guard never fires because $\text{over}[i]$ stops at $1 < 2$, and $\text{In}_i = M + 1$ against a claimed $B + k\Lambda(a_i) = 3$. Written out, the sharp forms are $\text{In}_i < B_{\max} + k\Lambda(s_i^P)$ and $\text{excess}_P[i] < B_{\max}/k + \Lambda(s_i^P) + (2 - 2/k)\Lambda(a_i)$, and since $\Lambda(a_i) \leq \Lambda(s_i^P)$ they give (4) with $\Lambda(s_i^P)$ throughout. The downward direction of Theorem 1 holds with $\Lambda(\max(s_i^P, s_i^{\text{FCFS}}))$, which we do not claim to be the smallest horizon that works, and Corollary 2 holds with $\Lambda(s_i^{\text{FCFS}} + G)$, since Lemma 2 bounds jobs that start as late as that; both are false with $\Lambda(a_i)$.
2. Servers may have different speeds. With speeds $v_1, \dots, v_k$ and $V = \sum_{\ell} v_{\ell}$, Theorem 1 holds with k replaced by V on the left and the same $2(k - 1)L$ on the right, and the guard satisfies $W_P[i] \leq W_{\text{FCFS}}[i] + B/V + (3k - 2)L/V$: the budget is divided by total capacity, not by the server count. Here L bounds a job's work in a common unit, not its wall-clock duration: a common wall-clock limit does not produce policy-independent executed work when speeds differ. The reference assigns each dispatched job to the fastest free server, ties by server index.
 3. The timeout is already in the model. Section 3 defines C_j as executed work, $\min(C_j, L)$ for a job stopped at the limit, so every currency above $(\text{In}_i, \text{Out}_i, \text{over}[q](t))$ and L is in executed work and no assumption on raw service demands is needed. Read in raw work the statements are false, a stopped job contributing more to In_i than is ever executed.
 4. Setup times survive. If a setup of length θ is paid before every service under both policies, absorb it into the job as $C_j' = C_j + \theta$; the model is unchanged and every result holds with $L \mapsto L + \theta$.
 5. Work conservation cannot be dropped. If servers may idle while jobs wait, let Γ_i^P be the server-idle time accumulated during $[a_i, s_i^P)$ while at least one job waited. Then (1) becomes $kW_P[i] = R_i^P + \text{In}_i - \text{Out}_i - \rho_i^P + \Gamma_i^P$, and nothing survives without a bound on Γ : with $k = 1$, pausing the server for T while i waits leaves $\text{In}_i = \text{Out}_i = 0$ and makes the excess T . Release constraints are the same phenomenon: if rank is keyed on arrival while eligibility starts later, the identity fails by exactly the forced idle time, and ranking on release repairs it.

6.10. A floor of order L , and what a guarantee costs

Proposition 3 (Impossibility). Fix any budget rule that assigns a positive budget to every waiting job at the first dispatch epoch. Then there are a base policy A that observes a job's service time only at its completion, and an input, such that Algorithm 1 run around A with that rule leaves some job with excess $\geq L$. For a base policy that ranks by predictions, such as SPJF-E, the predictions in the input are enough and A need not be chosen. Consequently no promise $G < L$ can be made by a wrapper of this family that holds for every base policy, or for one prediction-ranked base policy under arbitrary predictions, which is the generality in which Theorem 3 is stated. At $k = 1$ and constant budget $B > 0$, the bound $B + L$ of Theorem 3 is the exact supremum of the excess, approached and not attained; at $B = 0$ the wrapper is FCFS and the excess is exactly zero.

The proof puts $k + 1$ jobs of service L into an empty system at $t = 0$: nothing has completed, so no waiting job fires, the wrapper accepts the base policy's choices, and the first choice beyond the first k ranks displaces a job that first-come first-served starts at once, leaving it with excess L . The hypothesis can be neither dropped nor tightened: as a claim about policies the statement is false, since first-come first-served itself has excess 0 on every input, and a rule that is 0 at the first dispatch epoch and positive later falls to the same instance placed behind a saturating prefix. Supplementary Section S4.8 gives the proof and the case analysis that makes those cases exhaustive for the budget of Algorithm 1, which either makes the wrapper first-come first-served around every base policy or ad-

mits a base policy and an input with excess at least L , and Supplementary Section S8.4 gives the single-server family that approaches the bound $B + L$ without reaching it.

The guarantee also caps how much reordering can buy at all, and not only how much it may cost a single job.

Theorem 6 (Price of the guarantee). *Take m jobs of service L and n jobs of service $\sigma < L$, the long ones ranked first, all arriving at $t = 0$, and let P be any non-preemptive work-conserving policy with $\text{excess}_P[j] \leq G$ for every job. Define $D_0 = mn(L - \sigma)/k$, $E = 2(1 - 1/k)L(m + n)$ and $A = (kG + (3k - 2)L)/(n\sigma)$. If $D_0 > E$, equivalently $mn(L - \sigma) > 2(k - 1)L(m + n)$, then*

$$\frac{\overline{W}_{\text{FCFS}} - \overline{W}_P}{\overline{W}_{\text{FCFS}} - \overline{W}_{\text{SJF}}} \leq A + \varepsilon_+, \quad \varepsilon_+ = \frac{E(1 + A)}{D_0 - E}.$$

For fixed k, L, σ, G , the finite-sample term is $O((m + n)/(mn))$.

The proof is in Supplementary Section S8.5, which counts inverted pairs in two ways and so reads the pairwise-interchange argument behind Smith's rule [86] on an arbitrary arrival sequence. The right-hand side is capacity times guarantee, divided by the work that wants to overtake; it is linear in G and saturates at 1 when kG reaches the total service of the impatient class. Because it applies to every policy with the guarantee and not only to Algorithm 1, it describes the trade-off an operator faces rather than a weakness of this wrapper. The additive $(3k - 2)L$ in it is slack: it comes from applying Corollary 2 to each long job separately, where the long jobs can be charged together, and at $k = 1$ the aggregate argument in the same place removes it. There the achievable fraction is known exactly: it is $\lfloor G/\sigma \rfloor / n$, capped at 1, which Supplementary Section S8.5 states as its own proposition and checks exhaustively over 1,161 parameter settings.

6.11. Achievability and maximal permission

The preceding results could otherwise be read only as a conjunction of a robust wrapper and a necessary budget. The next theorem matches the price bound, up to its stated finite-sample band, on the two-size batch family; the theorem after it identifies the pointwise-maximal permission rule in a precisely restricted single-server class. Neither result gives instance optimality on arbitrary inputs, and the exact value of the optimum below remains open for $k \geq 2$.

Theorem 7 (Achievability on the two-size batch family). *In the family of Theorem 6, define*

$$\Delta(Q) = \sum_i (W_{\text{FCFS}}[i] - W_Q[i]), \quad F(Q) = \frac{\Delta(Q)}{\Delta(\text{SJF})},$$

and let $F^*(G)$ be the maximum of $F(P)$ over all non-preemptive, work-conserving policies satisfying $\text{excess}_P[i] \leq G$ for every job. Let the guard use exact SJF as its base and the constant budget $B = kG - (3k - 2)L \geq 0$. Put

$$D_0 = \frac{mn(L - \sigma)}{k}, \quad E = 2(1 - 1/k)L(m + n), \quad q = \min\left(1, \frac{B}{n\sigma}\right), \quad A = \frac{kG + (3k - 2)L}{n\sigma}.$$

If $D_0 > E$, then

$$q - \varepsilon_- \leq F(\text{guard}) \leq F^*(G) \leq \min(1, A + \varepsilon_+),$$

where

$$\varepsilon_- = \frac{E(1 + q)}{D_0 - E}, \quad \varepsilon_+ = \frac{E(1 + A)}{D_0 - E}.$$

Thus the leading terms are separated by at most $2(3k - 2)L/(n\sigma)$, in addition to the two displayed finite-sample errors. For $k \geq 2$, this sandwich does not determine the exact $F^*(G)$.

Definition 1 (Opaque-base, size-oblivious wrapper). *At a dispatch epoch, the wrapper may use arrivals and ranks, its own dispatch and completion history, the revealed sizes of completed jobs, the public bound L , fixed random coins, and the sequence of proposals made so far by an opaque base policy. It may not inspect unfinished job sizes or the base policy's internal state. A deterministic pathwise promise G means that every job has excess at most G for every finite input and every possible base-proposal transcript.*

Theorem 8 (Pointwise maximality at one server). *Assume $k = 1$, sizes and budgets are non-negative integer time units bounded by the positive integer L , and $G \geq L$ is an integer. Within Definition 1:*

1. *for constant budget $B \geq 1$, the guard's worst pathwise excess is exactly $B + L - 1$; at $B = 0$ it is FCFS and the excess is zero;*
2. *the largest positive integer constant budget with promise G is $B^* = G - L + 1$;*
3. *at any reachable history, if the queue head h satisfies $\text{over}[h] > G - L$, every wrapper with promise G must dispatch h . If $\text{over}[h] \leq G - L$, permitting any currently waiting later-ranked job for this dispatch is safe against the worst unrevealed size L . Consequently the guard triggered by $\text{over} \geq B^*$ is pointwise maximal in its permitted action set.*

Corollary 4 (The single-server consistency–robustness frontier). *On the integer two-size batch family, for $G \geq L$ the pointwise-maximal guard around the exact-SJF proposal sequence closes the fraction*

$$\min\left(1, \frac{\lfloor (G - L)/\sigma \rfloor + 1}{n}\right)$$

of the FCFS-to-SJF total-wait gap, and no opaque-base wrapper with the pathwise promise can permit a larger fraction of those short-before-long moves. Exact SJF itself reads unfinished true sizes, so the composed exact-SJF-plus-guard scheduler is not size-oblivious; the frontier concerns the wrapper's permissions under an externally supplied proposal transcript.

Supplementary Section S8.6 proves these statements. Supplementary theory material records an unbounded instance-optimality counterexample and the limits of instance-dependent refinements.

Every uniform constant above is on the scale of L . Proposition 3 establishes such a floor for the wrapper family under arbitrary opaque base proposals, not for all scheduling mechanisms. Empirically, usefulness depends on size dispersion, score predictability and the arrival structure; the CI pools are negative cases rather than evidence of a necessary distributional condition. Section 8.8 quantifies that boundary on the evaluated workloads.

7. Traces and Workload Characterisation

7.1. The Traces

The validation scenario is an educational one: two programming-assignment judging platforms supply the demand traces, and four further workloads bound the claim from outside that setting. What each trace can be used for is decided by whether its arrival times are observed or reconstructed, whether the queue wait is recorded, and how long a job may run there (Table 2).

Both CI pools and the compute farm carry a recorded queue wait. We validate the simulator against the CI pools, whose capacity is a countable set of physical machines that changes only on reimaging or quarantine. The second judging platform has no timestamp

Table 2. The traces, their role and what they record. “Arrivals” says whether the arrival instants are recorded, reconstructed proxies, or drawn; “Waits” whether the queue wait of each job is itself recorded.

Trace	Role	Size	Time fields	Arrivals	Waits
CodeBench [16]	main demand trace	4,611,325 blocks, 863,149 graded submissions in the development pool	one stamp per block, the end of the run	start proxy	no
ACcoding [17]	second judging platform	4,046,652 rows, 3,237,322 non-sealed	none; submission id only	no, drawn	no
Serverless invocations [7]	method applies outside education	1,980,951 invocations over 14.000 d	modified end stamp and duration	start proxy	no
CI pool A [20,21]	simulator validation, light tail	191,970 runs over 21 d	created, scheduled, started, resolved	yes	yes
CI pool B	second validation pool	30,702 runs over 6.99 d	as above	yes	yes
Compute farm [18,19]	non-applicable case	9,054,066 jobs over 30.00 d, 8,441,207 serial with positive run time	submit, wait, run	yes	yes
OULAD [87]	calendar-driven load elsewhere	9 course-presentations, daily counts	day	—	—

column, only a submission id, so job order inside a group and job sizes are real while every arrival instant is drawn. The correlation between a group’s first id and its recorded start time is 0.9995 over 469 groups, but in 207 of 468 consecutive transitions the id ranges overlap, so the id is an ordering and not a clock. Its results are a check that the ordering mechanism transfers, not a wait in seconds.

7.2. What the Fields of the Main Trace Mean

Several fields of that trace had to be interpreted before a queueing model could use them. Four readings carry the rest of the paper; Supplementary Section S2 gives the audit that establishes them, together with the release-statistics reconciliation and the two truncated archives.

The timestamp on an execution block is the moment the result was written, not the moment the code was received, so we take done_j to be the recorded stamp and reconstruct the arrival as $a_i = t_i - C_i$, carrying the opposite reading through the pipeline as a sensitivity with its own features and models. Stamps are recorded to the whole second while costs are recorded to the microsecond, which opens a leak that we close by jittering each stamp deterministically: a classifier given only the fractional part of the gap to the previous record separates heavy from light jobs with AUROC 0.8091 on the whole-second clock and 0.4499 after the jitter, while the forward models of the two clocks stay within 0.008 of AUROC of each other. The observed regime has an upper ceiling of 60 s; this is not evidence of a documented enforced timeout. The last run to reach a longer time ends at 2019-08-28; the periods before that date stay out of the primary pool, and where used at all they carry the later limit, $C^{\text{cap}} = \min(C, 60)$ (Supplementary Table S3). Groups of runs by the same user that end in the same second are 7.53% of blocks and 26.28% of capped work; they overlapped and stretched each other, so we neither delete them silently nor treat each as an independent arrival, and the scheduling trace is rebuilt twice, once with each group collapsed into one job and once with the groups removed (Supplementary Table S14).

7.3. What the Source Platform Identifies

The platform that produced the main trace gives each logged-in student a dedicated container and runs their code inside it. Its logs do not identify the waiting times of a shared judging queue. We use recorded durations and reconstructed start-time proxies as

inputs to a hypothetical shared pool; its capacity, queue and service order are our design. Neither this construction nor the absence of shared-queue fields establishes that all source-platform work experienced no waiting.

Three things support the design. The first is that a fixed pool of judging hosts on one queue is a standard deployment. A widely used contest judging system recommends “one judgehost per 20 teams” [2] on dedicated machines, and the autograder study of Section 1 runs a configurable number of concurrent graders, replays the busiest hours before a shared deadline and measures the queue wait [3]. That replay compresses its window fourfold and sixteenfold; we raise load by overlaying whole class-terms instead (Section 7.4), for the reason Section 9 gives. Dedicated per-entity capacity is also known to sit idle in the workloads this one resembles. In the serverless characterisation behind our invocation trace, 45% of applications are invoked once an hour or less and 81% once a minute or less, so the authors report that the cost of keeping them warm relative to their billable execution time can be prohibitively high [88]; and a cluster manager that measured the alternative directly reports that splitting each large user’s workload into its own cell would need 2 to 16 times as many cells and 20 to 150% more machines [89]. Neither source is about judging, and neither licenses a number of our own.

The second support is a measurement on the same demand: replayed under the dedicated design, the 99th percentile of containers alive at once is 1,636 and each container executes code for 0.12% of the time it is alive, while a shared pool holds the 99th percentile of deadline-window waits under 1 s with 14 servers under FCFS and 13 under the predictor-ordered policy. The third is that the simulator reproduces waits somebody else recorded: the CI logs carry the queue wait of every run, and an effective capacity and a setup term refitted on one chronological part of each window return the held-out mean wait, the per-hour profile and the per-job ordering, with the 90th percentile the one quantile that does not survive. Supplementary Section S2 gives both in full, with the six session reconstructions, the five chronological splits, the step-of-one sharpness check and the circularity of the original whole-window fit. The pooled side is simulated, so neither supports a claim that the platform ever queued.

Capacity and physical checks.

The supplementary capacity sweep retains the complete offered stream and varies the number of servers; it uses the archived embedded Tweedie fit without refitting. The physical experiment executes FCFS, SPJF-E and a guard on the complete selected development window, with timed slot occupancy and recorded enqueue, dispatch and completion events. It tests an executing rule but does not transport submitted programs to new hardware. The selected empty-start window is distinct from a full-trace replay; its guarded p99 is worse and its job-level ideal-simulator agreement fails (Section 8.9, Supplementary Section S6).

7.4. Building the Shared Pool

A single class-term does not load one server, so we overlay several. The pool takes only the development terms in the observed 60 s regime, 40 class-terms in all, and replicates them 44 times, each copy aligned by weekday and hour and shifted by a whole number of weeks drawn per copy. Overlaying whole terms rather than compressing inter-arrival times preserves the daily and weekly cycle and each job’s position relative to its own deadline. The result is 17,634,760 jobs over 30 weeks and 208.8 days. The three load levels share that trace and differ only in the pool size. Four of the five overlays take $k = 8/5/4$ and the fifth takes $7/5/4$, its work per busy hour being the lowest of the five; on the first overlay that gives busy-hour utilisations of 0.529, 0.847 and 1.059, and 0.502,

0.783 and 0.979 averaged over the five. A deadline window is the 24 hours before an assessment's own end, and 4,219,556 jobs arrive inside one. Five overlays are week-shifts of the same terms, so they are one construction and not five platforms, and our intervals resample whole weeks, which covers time and leaves the choice of terms uncovered. A trace built from the last development term alone is not available this way, since it would need 190 copies of its 8 class-terms to load a pool of four.

7.5. What the Load Looks Like

Arrivals follow the course calendar. Under grouped-class cross-validation, predicting executions per class 168 hours ahead gives log-scale RMSE 1.243 with lag features alone, 1.159 with calendar features alone, and 1.112 with both. A paired bootstrap over class-week blocks gives a combined-minus-lag-only difference of -0.1307 $[-0.1395, -0.1213]$. The corroborating trace shows the same shape elsewhere: clicks in the three days up to an assessment due date are 1.29 to 2.95 times the other test days across its nine course-presentations, and a seven-day-ahead peak warning on distinct active students goes from F1 0.000 under a moving-average baseline to 0.593 once calendar features are added.

Costs are concentrated in a small fraction of jobs on every workload except the CI pools, which motivates the empirical cost-concentration screen (Supplementary Table S3, which also records the provenance of the analysis bound L on every trace). Supplementary Section S2 gives what the arrival and service fields of each trace are, which of them are proxies, and how many jobs each analysis truncation touches.

7.6. Sealed Data

Three of the traces carry a part that was sealed before the method was designed. On the main trace it is three terms, 2023-1, 2023-2 and 2024-1; on the second judging platform it is the last 20% of the submission-id range, 809,330 rows, of which only the row count and the first id have been read; on the corroborating trace it is the 2014 academic year. Only the main trace's sealed terms will be opened for this paper, once, after the method, its parameters and the primary metric have been frozen; the frozen pipeline covers those three terms and nothing else, and the sealed parts of the other two traces stay shut and are reported on by neither. The serverless trace, the compute farm and the two continuous-integration pools have no sealed part, so every figure we give for them is development data. No cost distribution, prediction or scheduling result has been computed on any sealed part, and every access to one is entered in a ledger in the repository. Every number in this section and in Sections 4 and 5 is a development number. The sealed evaluation remains pending in this manuscript; Section 8.2 states the current output scope.

8. Experiments

The experiments are organised by the three questions of Section 1. Question 1, whether the cost of a job can be estimated before it runs, is answered in Section 4 and is not repeated here. This section takes the predictors as given and asks what a scheduler should sort by (Section 8.4), what the per-job guarantee costs and what it buys (Section 8.5), whether the bounds of Section 6 hold on real traces (Section 8.6), and where the method applies (Sections 8.8 and 8.10).

8.1. Setup

The main scheduling comparisons replay the overlaid CodeBench pool built in Section 7.4. The three load levels differ only in the pool size and the five overlays only in the per-copy week shift, and every policy sees the same arrival instants and the same service times, so the comparison is pathwise. Supplementary Table S4 collects the configuration and the selection protocol; the metrics are defined in Section 3 and the policy names

in Table 1, extended by the aging comparator below. Section 8.3 gives the conservative-visibility headline. The original-score ranking, budget-shape, corrupted-score, residual and single-server results retain the source result clock and are optimistic references, not estimates of the conservative protocol. The capacity and physical experiments use an archived development fit and are reported separately. We report the fraction of the FCFS-to-SJF gap closed and the plain percentage reduction against FCFS side by side throughout, because they are different numbers and the first is easy to read as the second.

8.2. What Was Selected on What

Three statements precede the numbers. The guard family and a coarse grid over (B_0, η) were explored on the evaluation trace before any validation-only selection existed, so the selection chooses inside a family already narrowed on the trace this section reports. Nothing here is held out: the validation pool is the development terms minus the last one, which is held out of the selection and not of the development process, and which cannot load a pool on its own (Section 7.4), so we describe no figure in this section as out of sample, validated on test data, or evidence of generalisation. And the grids, the feasibility rule, the objective and the tie-break were fixed before the grids were run, at comparable density across the three shapes, with exact integer microseconds for arrivals, service and budget arithmetic. Supplementary Section S5 records what the selection chose, which results the released package reproduces and which remain archived development evidence, and the drop-one-cell sensitivity of the worst-cell rule. No sealed result is printed here.

8.3. Outcome Visibility in the Replayed Queue

The original feature sweep releases an outcome at its source-platform done_j , whereas under policy P the replayed job completes at $a_j + W_P[j] + C_j$, so a feature may be causal in the recorded log and still use a result unavailable in the replay; neither a future-record perturbation test nor a source-clock brute-force feature audit resolves that. The conservative score therefore restricts user, exercise and pair history to submission outcomes in the same class-term copy and admits a result only when $\text{done}_j + 3600 \leq a_i$, while the static score uses code, course and clock information known at submission, without outcome history. Both use their own fitted models, so the three variants differ in more than release delay and their differences cannot be attributed to queueing delay alone. Guard parameters remain those selected under the original score, and the conservative variant is the empirical headline. Supplementary Section S3.7 gives the same-copy exposure decomposition, the lag audit that finds 0 violations in every conservative Guard and FCFS cell and 336 job occurrences beyond the lag under unguarded conservative SPJF-E, the pending policy-specific exact variant, the aging comparator outside the budget family and the package predictor metrics with their splits.

At the busiest load, conservative Guard(600) gives p99 174.71 s and closes 0.429 [0.357, 0.533] of the reference gap, against 89.24 s and 0.801 [0.764, 0.842] under the original score. The static control closes 0.349 [0.281, 0.457]; the paired conservative-minus-static gap is 0.080 [0.039, 0.124]. Thus retained history improves on the static control here, but the original-score effect is substantially larger. These intervals condition on the constructed development overlays and do not cover semester selection, model refitting or deployment variation.

8.4. Question 2: What to Rank By

Table 4 gives the unguarded policies at the three loads. Ranking by expected cost is ahead of the log-scale score at the two higher loads and unresolved at the lightest: the paired difference in gap closed is $-0.000 [-0.016, 0.013]$ at $\rho = 0.5$, $+0.012 [0.004, 0.019]$ at $\rho = 0.8$ and $+0.019 [0.011, 0.024]$ at $\rho = 1.0$. The control for refit noise, the same

Table 3. Guard(600) under unchanged parameters and different information protocols. Conservative is the headline; original is an optimistic reference. Gap intervals are paired week-block intervals. Waits, excess and harm are in seconds.

Load	Score history	p99 _{dl}	Gap closed	Max exc.	Harm	Fired (%)
$\rho = 0.5, k = 7.8$						
	original	14.97	0.740 [0.713, 0.760]	492.2	237.0	0.17
	conservative	16.68	0.640 [0.604, 0.671]	499.9	235.2	0.94
	static	17.95	0.560 [0.519, 0.594]	498.0	225.1	0.38
	exact (pending)	—	—	—	—	—
$\rho = 0.8, k = 5$						
	original	47.47	0.837 [0.815, 0.851]	507.8	319.6	1.31
	conservative	64.77	0.634 [0.560, 0.688]	509.4	327.7	3.26
	static	70.18	0.571 [0.504, 0.638]	512.9	341.6	3.89
	exact (pending)	—	—	—	—	—
$\rho = 1, k = 4$						
	original	89.24	0.801 [0.764, 0.842]	523.7	339.9	7.15
	conservative	174.71	0.429 [0.357, 0.533]	527.8	303.5	7.71
	static	193.11	0.349 [0.281, 0.457]	528.1	316.1	11.18
	exact (pending)	—	—	—	—	—

log-scale estimand refitted from scratch, moves the gap by +0.001 [−0.002, 0.002], −0.000 [−0.001, 0.001] and −0.000 [−0.001, 0.000], so the advantage is larger than the noise of re-fitting and is not produced by it; that control and the sequence state come from the earlier development study and are the two rows of Table 4 the package does not produce. The sequence state of Section 4 does not translate into a consistent gain and we do not claim it as an improvement.

Alternative ranking losses fitted on the same features and run through the same simulator are Supplementary Table S13: the fitted quantile scores and the raw-second squared-error model lose to the Tweedie fit at every load, and a two-class rule that only separates predicted-heavy from predicted-light jobs recovers little of the benefit, so most of it comes from the fine ordering inside the light class.

The two kinds of error do not cost the same. Replacing the largest 1% of underestimates by the true cost, and leaving the rest of the score alone, closes 0.91 of the distance to the SJF reference in the deadline-window p99 at $\rho = 1.0$ on both overlays, where repairing the largest 1% of overestimates instead closes 0.26 and 0.24, and at 0.1% the two figures are 0.47 and 0.04; Supplementary Section S3.16 gives the six waits behind them. Treating a heavy job as light and letting it run first is the expensive mistake, by about an order of magnitude. This diagnoses the fitted predictor: replacing a score by true cost depends on its numerical scale and is not invariant to a monotone score transformation. It neither establishes a universal error asymmetry nor distinguishes the population targets of raw squared error and Tweedie loss.

8.5. Question 3: What the Guarantee Costs and Buys

Table 5 reports the guarded policies at three promises and three loads, and Supplementary Figure S2 plots the gap closed against the promise. The cost of the promise depends on the promise and on the load. Under the original source-clock score, an optimistic reference rather than the headline of Table 3, at $\rho = 1.0$ Guard(600) closes 0.801 of the FCFS-to-SJF gap against 0.916 for the unguarded policy, so it gives up 0.115 of the gap at a promise of ten job-limits, about an eighth of what the unguarded ordering buys; at $G = 1200$ s the cost is 0.087 and at $G = 300$ s, five job-limits, 0.438. The paired week-block intervals for these three differences are [−0.142, −0.077], [−0.098, −0.064] and [−0.526, −0.314]; none contains zero, so the promise has a resolved cost at every G we tested.

Table 4. Unguarded policies, five overlays. Waits in seconds; gap and reduction on the deadline-window p99, with 95% paired week-block intervals. Original source-clock scores; these are optimistic references.

Load	Policy	p99 _{dl}	Gap closed	Red. (%)	Mean
$\rho = 0.5, k = 7.8$	FCFS	27.64	0.000	0.0	0.719
	SJF	10.59	1.000	62.0 [58.7, 66.4]	0.228
	SPJF-log	14.71	0.754 [0.722, 0.777]	46.7 [43.9, 50.1]	0.296
	SPJF-log, refitted ^a	14.72	0.753 [0.722, 0.776]	46.7 [44.0, 50.0]	0.295
	SPJF-E	14.68	0.754 [0.725, 0.776]	46.7 [43.9, 49.9]	0.292
	SPJF-E, GRU state ^a	14.87	0.741 [0.707, 0.767]	45.9 [43.2, 48.8]	0.290
$\rho = 0.8, k = 5$	FCFS	119.35	0.000	0.0	4.160
	SJF	33.55	1.000	71.8 [67.0, 75.1]	0.959
	SPJF-log	44.36	0.873 [0.852, 0.887]	62.7 [57.5, 66.2]	1.252
	SPJF-log, refitted ^a	44.41	0.873 [0.851, 0.886]	62.7 [57.4, 66.2]	1.251
	SPJF-E	43.33	0.885 [0.862, 0.899]	63.6 [58.1, 67.4]	1.232
	SPJF-E, GRU state ^a	42.82	0.891 [0.870, 0.904]	64.0 [58.5, 67.7]	1.230
$\rho = 1.0, k = 4$	FCFS	273.50	0.000	0.0	10.270
	SJF	43.63	1.000	84.0 [79.6, 86.8]	1.696
	SPJF-log	67.17	0.897 [0.872, 0.914]	75.4 [70.2, 78.6]	2.354
	SPJF-log, refitted ^a	67.36	0.896 [0.871, 0.913]	75.3 [70.1, 78.5]	2.352
	SPJF-E	62.79	0.916 [0.892, 0.930]	77.0 [71.6, 80.3]	2.294
	SPJF-E, GRU state ^a	62.71	0.917 [0.895, 0.929]	77.0 [71.7, 80.3]	2.302

^a These two rows are not produced by the released package, which fits only the expected-cost and log-scale scores. They come from the earlier development study (evidence/main_v3), which used a floating-point clock and a stored copy of the predictions, and are printed at the values that study reported; the remaining rows differ from it in the second decimal of a wait.

The validation harm constraint need not transfer to new cells. The original-score Guard(600) has evaluation harms of 319.6 and 339.9 s that exceed the 300 s validation limit at the middle and busiest loads. This is failure of that empirical constraint to transfer, while every excess remains within the proved promise. Supplementary Figure S1 displays each selected shape and the full-cap comparator against harm, with the validation trade-offs shown separately, and Supplementary Section S3 makes the comparison with the aging baseline under the same conservative protocol.

The jobs that pay are the heavy ones, and the worst wait among them is the last of the secondary metrics of Section 3. The guard leaves the worst heavy job inside the promise at every load, within about 450 s of the FCFS figure, where the unguarded ranking multiplies it by between 2.8 and 4.1; Supplementary Section S3.16 gives the nine waits.

What the shape of the budget is worth.

The promise is the cap's alone, so the three shapes are compared at a fixed promise and the question is only how much of the base policy's benefit each keeps and who pays for it. The shape matters where the promise is tight and the load is high, and hardly at all elsewhere; it decides which jobs pay for the promise more than it decides how much of the benefit survives. Supplementary Section S3 gives the paired differences at the three promises, the harm axis on which the queue-length shape comes within 0.009 of the joint winner's gap at under half its worst harm, and the equal-promise Fixed(G) comparator that the harm rule rejects at every promise while leading on the gap in eight of the nine cells.

Counting positions buys almost nothing on this load (Supplementary Figure S2, dotted). This measures how loose the count bound is here, not whether a count can give a bound at all: (2) does give one, and the count rule is run at the largest count that bound allows at each promise. What makes it loose is the cost distribution. Overtakers on this trace are overwhelmingly shorter than a second, while a count of κ has to be sized against

Table 5. The guard at three promises and the comparators. Guard(G) is the shape the selection rule picks over all three grids and Guard-fixed(G) is the winner inside the constant grid; the age-relative and queue-length winners are Supplementary Table S6, and one of the three repeats Guard(G) at each promise. Fixed(G) spends the whole cap at the same promise and is infeasible under the harm rule. Budgets scale as $k/4$ and the skip count is $\kappa = \lfloor Gk/L \rfloor - (2k - 2)$, the largest integer with $(\kappa + 2k - 2)L/k \leq G$, so both differ with the cell's k : at $\rho = 0.5$ the four overlays at $k = 8$ run $\kappa = 26/66/146$ and the fifth, at $k = 7$, runs $23/58/128$; the column prints the second of the two. Excess and harm are the worst over the five overlays, the rest are means. All rows use original source-clock scores (optimistic reference).

Load	Policy	Promise	p99 _{dl}	Gap closed	Red. (%)	Max exc.	Harm	Fired (%)
$\rho = 0.5, k = 7.8$; FCFS p99 _{dl} = 27.64 s, SPJF-E 14.68 s with max excess 1,241.4 s								
	Guard(300)	300	15.58	0.705 [0.678, 0.724]	43.7	192.5	132.1	1.35
	Guard-fixed(300)	300	15.75	0.697 [0.673, 0.717]	43.2	143.0	132.1	2.12
	Fixed(300)	300	15.30	0.721 [0.696, 0.738]	44.7	193.0	181.8	0.77
	Skip(300), $\kappa = 23$	300	26.38	0.078 [0.052, 0.107]	4.8	62.1	59.6	45.41
	Guard(600)	600	14.97	0.740 [0.713, 0.760]	45.8	492.2	237.0	0.17
	Guard-fixed(600)	600	15.09	0.733 [0.706, 0.752]	45.4	298.7	237.0	0.36
	Fixed(600)	600	14.91	0.742 [0.715, 0.763]	46.0	493.3	237.0	0.11
	Skip(600), $\kappa = 58$	600	24.64	0.184 [0.135, 0.233]	11.5	100.2	63.3	37.26
	Guard(1200)	1200	14.84	0.746 [0.719, 0.768]	46.2	907.8	94.3	22.01
	Guard-fixed(1200)	1200	14.83	0.747 [0.719, 0.768]	46.2	557.8	237.0	0.06
	Fixed(1200)	1200	14.70	0.753 [0.724, 0.775]	46.6	1,077.0	237.0	0.00
	Skip(1200), $\kappa = 128$	1200	21.62	0.367 [0.288, 0.432]	22.8	102.3	67.6	28.80
$\rho = 0.8, k = 5$; FCFS p99 _{dl} = 119.35 s, SPJF-E 43.33 s with max excess 3,044.5 s								
	Guard(300)	300	55.94	0.738 [0.697, 0.761]	53.0	209.7	114.1	11.57
	Guard-fixed(300)	300	64.44	0.640 [0.543, 0.699]	45.9	171.6	151.1	20.62
	Fixed(300)	300	53.83	0.763 [0.721, 0.785]	54.8	210.9	195.4	10.75
	Skip(300), $\kappa = 17$	300	119.20	0.002 [-0.001, 0.009]	0.1	64.2	58.8	80.53
	Guard(600)	600	47.47	0.837 [0.815, 0.851]	60.1	507.8	319.6	1.31
	Guard-fixed(600)	600	49.85	0.809 [0.786, 0.824]	58.1	315.4	285.7	4.72
	Fixed(600)	600	47.03	0.842 [0.824, 0.853]	60.5	502.6	447.1	1.16
	Skip(600), $\kappa = 42$	600	118.69	0.008 [-0.002, 0.026]	0.6	97.5	63.5	74.02
	Guard(1200)	1200	46.85	0.844 [0.823, 0.857]	60.6	1,104.9	117.6	9.93
	Guard-fixed(1200)	1200	46.76	0.845 [0.826, 0.857]	60.7	581.8	469.2	0.86
	Fixed(1200)	1200	45.32	0.862 [0.842, 0.874]	61.9	1,106.3	469.2	0.22
	Skip(1200), $\kappa = 92$	1200	117.48	0.022 [0.004, 0.074]	1.6	162.5	83.6	66.10
$\rho = 1.0, k = 4$; FCFS p99 _{dl} = 273.50 s, SPJF-E 62.79 s with max excess 5,904.5 s								
	Guard(300)	300	163.40	0.478 [0.387, 0.602]	40.2	229.1	134.8	28.64
	Guard-fixed(300)	300	223.69	0.217 [0.151, 0.359]	18.2	165.1	158.4	41.50
	Fixed(300)	300	169.29	0.453 [0.369, 0.619]	38.1	222.6	208.2	28.33
	Skip(300), $\kappa = 14$	300	273.14	0.001 [-0.001, 0.003]	0.1	81.3	81.3	90.36
	Guard(600)	600	89.24	0.801 [0.764, 0.842]	67.3	523.7	339.9	7.15
	Guard-fixed(600)	600	116.75	0.682 [0.601, 0.764]	57.3	325.4	292.9	17.50
	Fixed(600)	600	86.53	0.813 [0.778, 0.851]	68.3	515.8	465.5	7.23
	Skip(600), $\kappa = 34$	600	272.98	0.002 [-0.004, 0.006]	0.2	112.4	82.4	86.25
	Guard(1200)	1200	82.67	0.830 [0.801, 0.857]	69.7	1,110.4	144.8	6.21
	Guard-fixed(1200)	1200	83.32	0.827 [0.798, 0.860]	69.5	576.5	522.7	5.76
	Fixed(1200)	1200	72.94	0.872 [0.846, 0.893]	73.3	1,112.9	708.8	0.73
	Skip(1200), $\kappa = 74$	1200	273.08	0.002 [-0.009, 0.013]	0.1	169.7	105.0	80.69

the worst case κL , so the count is exhausted long before the work it stands for is, and the baseline forces most dispatches back to the queue head. It is charged at dispatch; Supplementary Section S3.16 gives the passes that allows and what charging at completion instead would change.

Corrupted scores separate what the guard does from what it does not do (Supplementary Table S7, Supplementary Figure S3). Under a reversed order, a random order, or a score that calls the truly longest 1% the shortest, the unguarded policy delays some job by more than 6,000 s beyond FCFS at full load, and Guard(600) holds every job inside its promise. The deadline-window p99 is not rescued: it stays at 546.5, 261.9 and 543.6 s against 273.50 s for FCFS, which is worse than FCFS for two of the three. The guarantee is per job and additive, so this is what it permits: a guarded run may have a worse quantile than FCFS, by at most G , and what it removes is the worst single job and not the shape of the tail.

8.6. The Bounds on the Traces

Every guarded run asserts the per-job bound of Section 6 for each job separately, inside the simulation rather than afterwards on summary statistics. Across the 315 policy-by-cell runs reported here, that is 5,554,949,400 per-job checks with 0 violations, and the single-server configuration adds 21 runs and 67,396,287 checks, also with 0 violations. An independently written simulator, which shares no code with ours, reproduces the runs job for job on 17,634,760 jobs and 22 policies with 0 differing jobs, and asserts the same bounds on its own waiting times over 5.7 billion checks with 0 violations; that cross-check was run against the earlier development pipeline, whose dispatch sequence the package reproduces.

The ratio of the realised worst excess to the bound falls as k grows, from 0.9950 at $k = 1$ to 0.8850 at $k = 8$, while the finite-skip bound is attained exactly on small instances. These are facts about the traces and leave the constant where Section 6 put it: $(3 - 2/k)L$ is the exact supremum at every k , but the family that approaches it is a geometric cascade that needs hundreds of jobs arranged against the wrapper, and nothing in a replayed workload arranges them. Supplementary Section S3.16 gives the ratio at each k .

The identity, measured job by job.

Theorem 1 is asserted on every job of every cell of overlay 0: 0 violations over 1,428,415,560 job-checks, the residual $D_i = k(W_P[i] - W_{FCFS}[i]) - (\text{In}_i - \text{Out}_i)$ reaching at most 0.5173 of its ceiling $2(k - 1)L$ over all 81 cells, so that it never exceeds 0.518, which is about $(k - 1)L$. That is where a search which does not build the cascade of Theorem 2 stops, so the measurement says the workload does not attain the constant and not that the constant is loose. Table 6 reports four cells and Supplementary Table S8 all nine; Supplementary Section S3.13 gives the recording kernels with their validation, the same-phase shares and the variance of the excess that $(\text{In}_i - \text{Out}_i)/k$ explains.

8.7. A Single Server

At $k = 1$ the additive constant $(3 - 2/k)L$ is exactly L , the identity of Section 6 holds with no error term, and the separation between budget shapes is largest (Supplementary Table S9). Guard(600) and Fixed(600) are statistically indistinguishable on the gap closed and differ by a factor of 3.41 in harm; the corresponding factors are 3.01 at $G = 300$ s and 12.18 at $G = 1200$ s. The realised excess reaches 595.5 s against the promised 600 s, and under the worst corrupted score 597.0 s, so at one server the promise is close to being spent.

The single-server configuration also shows the cost of choosing a shape on one set of cells: the parameters were selected on the 15 multi-server validation cells, and $k = 1$ is not

Table 6. The residual D_i of Theorem 1 on overlay 0, 17,634,760 jobs per cell. Ratio is $\max |D_i|$ over the bound $2(k-1)L$, which is $14L$, $8L$ and $6L$ at the three loads; R^2 and the worst error are for $(\text{In}_i - \text{Out}_i)/k$ against the realised excess over FCFS. The last row is the reversed score at the busiest load, against the Guard(600) row above it: the residual is a property of the schedule and barely moves with the policy. Supplementary Table S8 gives all nine cells.

Load	Policy	$\max D_i /L$	Ratio	$D_i = 0$	R^2	worst err. (s)
$\rho = 0.5, k = 8$	Guard(600)	6.788	0.485	95.6%	0.983	50.9
$\rho = 0.8, k = 5$	Guard(600)	3.922	0.490	87.2%	0.995	47.1
$\rho = 1.0, k = 4$	Guard(600)	3.082	0.514	81.1%	0.998	46.2
$\rho = 1.0, k = 4$	Reversed score	2.994	0.499	81.7%	1.000	44.9

among them, so the joint winner transfers at the two tighter promises and not at $G = 1200$ s (Supplementary Section S3.14). We report the number the pre-stated rule produces and do not re-tune it on this cell; Section 9 states what it means.

8.8. Other Platforms

Table 7 places the main trace beside the other scheduling workloads under one format, including the supplementary LPC-EGEE counterfactual. The first three columns describe the workload, the next two what ranking by prediction does to it without a guard, and the last two the gap the guard closes and the excess it caps at a promise of five job-limits. Excesses are expressed in units of the per-job limit L , the unit the guarantee is written in.

Table 7. Cross-domain development comparisons. Tail share is the fraction of executed work carried by the longest 1% of jobs; AUROC is for the heavy class from arrival-time information; excesses are in units of the per-job limit L of that trace. “Cond. 1” is the first applicability condition of this section, that the first-come first-served p99 wait exceed the floor $(3 - 2/k)L$, which is close to $3L$: four of the original six workloads fail it, and on those four the uniform certificate is coarser than the wait itself. This is a usefulness screen for that certificate, not an impossibility claim for other policies. The CodeBench row uses original source-clock scores (optimistic reference).

Trace and load	Tail share	AUROC	FCFS p99/ L	Cond. 1?	Gap SPJF-E	Unguarded max exc.	Guard at $G \approx 5L$ gap closed / max exc.
CodeBench, busy-hour $\rho \approx 1.0$	0.375	0.936	4.56	yes	0.916	98.4L	0.478 / 3.82L
ACcoding, drawn arrivals, $\rho = 0.8$	0.122	0.927	0.021	no	0.980	20L	0.974 / 2.21L ^a
Serverless, busy-hour $\rho = 1.0$	0.443	0.985	4.22	yes	0.858	10.07L	0.710 / 2.52L
CI pool A ^c , $k_{\text{eff}} = 156$ of 173	0.034	0.651	1.78	no	n/a ^b	11.53L	0.735 ^b / 2.64L
CI pool B ^c , $k_{\text{eff}} = 96$ of 105	0.052	0.997	0.373	no	0.958	0.34L	0.958 / 0.34L
Compute farm, window $\rho = 0.95$	0.372	0.890	0.136	no	−4.111	0.85L	−4.346 / 0.83L
LPC-EGEE, chronological test ^d	0.351	0.892	0.042	no	0.193	0.162L	0.193 / 0.162L
LPC-EGEE test class, per-class pool ^e	0.213	0.746	13.76	yes	0.679	86.07L	0.569 / 3.48L

^a ACcoding arrival instants are drawn, not recorded, so every scheduling figure in this row is a statement about the ordering mechanism and not about waiting on that platform; its budgets were set at fixed multiples of L instead of from a promise, and the entry shown is the largest one tested, $B = 10L$. Its tail share is the simulated one, with the assumed fixed term and $L = 30.5$ s; on measured cost alone, at $L = 30.0$ s, it is 0.275 (Supplementary Table S3), the figure the introduction quotes. ^b On CI pool A, SJF is itself worse than FCFS at the p99, so the p99 gap has no usable denominator; the guard entry is the mean-wait gap. ^c (on the two k_{eff} row labels) The CI capacities are fitted, not recorded, and the two scheduling columns and the FCFS p99 of these rows exist only at the whole-window fit ($k_{\text{eff}} = 156$ of 173 and 96 of 105), so they are printed at it. What supports them is the held-out refit of Section 7, which recovers the same capacity to within a few machines on chronological splits of each window and reproduces the recorded waits on the untouched part, with the p90 the one quantile that does not survive. The tail share and the AUROC do not depend on the capacity. ^d LPC-EGEE [90] uses disjoint pools with 83 and 56 servers fitted on the earlier part; full chronological test wait-profile validation is weak. L is the largest documented queue limit. Its p99 and gaps use frozen-threshold busy hours, not deadlines; history follows recorded completions. The displayed age-relative guard at $G = 5L$ never fires. This conditional FCFS counterfactual is not evidence of real-scheduler improvement (Supplementary Section S6). ^e The test class is a pool we impose on machines the six queues demonstrably shared, at the boundary capacity $k = 1$ of the search grid on each partition, and its held-out wait profile is not reproduced. Condition 1 holds on both partitions’ recorded waits and on the pooled FCFS counterfactual, and fails at the $3L$ form on one partition and at the observed concurrency ceiling on both. The 0.679 is the unguarded score’s busy-hour p99 gap, whose interval spans zero, and the guard’s own is −0.103, so the last column reports its resolved mean gap, 0.569 [0.534, 0.630], at $G = 3.5L$ rather than the $5L$ of the other rows, where its largest excess is 3,129 s of an allowed 3,150 s (Supplementary Section S6.6).

Two empirical screens assess where reordering and the uniform certificate may be useful. The first compares the FCFS tail scale with the certificate's additive term $(3 - 2/k)L$: four of the original six workloads have smaller FCFS p99 values, which means the certificate is too coarse at that scale and not that finer promises are impossible for a restricted base policy or workload. CodeBench at $4.56L$ and the serverless trace at $4.22L$ clear it; the compute farm is the extreme, with an FCFS p99 of 18.7 h against a bound $L = 493,857$ s, or 5.7 days, and guards that never fire. The second screen is cost concentration, an opportunity for reordering rather than a necessary condition: in the CI pools the longest 1% carry only 3.4% and 5.2% of work, and true-size ordering worsens p99 on three of the four reported windows. Where both are cleared, in the original-score CodeBench and serverless comparisons, the guard closes 0.48 to 0.71 of the reference gap and limits observed excess to 2.5 to $3.8L$ at a promise of $5L$. Clearing either screen does not guarantee a percentile improvement: the capacity experiment below contains counterexamples.

Which screen a trace clears can depend on how its pool is drawn. Read as one pool per walltime class rather than one per site, LPC-EGEE's 900-second class has an FCFS p99 of $13.76L$ and is the one reading of that log on which the work-budget guard fires, spending 0.993 of its allowance against 0.046 pooled, while cost concentration inside it falls to 0.213 from the 0.407 of the pooled trace; the row's footnote and Supplementary Section S6.6 say why that is not evidence of deployment benefit.

8.9. Capacity and Physical Implementation

The supplementary capacity sweep tests the archived score at every integer $k = 1, \dots, 20$ on all five development overlays and extends the first overlay to its exact zero-wait endpoint $k = 76$. Benefit changes sign with capacity: at $k = 1$ Guard(600) worsens deadline p99 by 493.79 s, while positive simultaneous support for its improvement occurs only at $k = 4$ –12, so the selected load points are not evidence of positive benefit at every capacity.

The physical timed-work experiment executes 2,151 jobs on two workers in a selected empty-start release window. All 6,453 observed-prefix decisions pass the rule audit, but the guard's per-job discrepancy from its ideal measured-input replay reaches 1,354.10 s and its physical p99 is 1,783.65 s against 1,763.30 s for separately timed FCFS, so this window does not establish a guarded p99 gain. Supplementary Section S6.2 reports the positive execution checks beside these negative results.

8.10. Sensitivities

Five trace-construction variants were run through an earlier source-clock pipeline, so they are sensitivity results for its stored M4 log-scale score and fixed completed-work budget and not for the selected expected-cost Guard(G): within that configuration the alternatives retain a positive gap while the earlier fixed-budget guard ranges from 0.442 to 0.843 of the gap at full load. They support sensitivity of the earlier ordering experiment to timestamp interpretation, source-term inclusion and co-ending-run treatment, and validate neither the final selected Guard(G) nor conservative history nor policy-specific exact visibility. Supplementary Table S14 gives the five variants and Supplementary Section S5 the configuration and release boundary.

A sixth variant changes the arrival process instead of the trace construction. The logs carry a recorded think time, the offset between one result and that student's next submission, and releasing each submission after its predecessor's result plus that offset shortens every wait by a factor of 1.3 to 3.8. The baseline gains most, its own longer waits delaying its own users more, so the absolute gap narrows while the fraction of it a policy closes does not: Guard(600) closes 0.825, 0.809 and 0.810 of it against 0.734, 0.827

and 0.800 at the recorded instants, and the price of the promise falls with the queue, from 41.6% above unguarded SPJF-E to 3.7%, while the guarantee is untouched on all 17,634,760 jobs of each run against first-come first-served on that run's own realised arrivals. These runs read the original-clock scores of the main development cells, so they are a sensitivity to the arrival process and not a restatement of the conservative headline; Supplementary Section S6.5 gives the table, the bootstrap and the limits, the first being that the recorded think times were produced on a platform where no student waited.

Two further limits of these runs — what the intervals do and do not cover, and the prediction cost the runs do not charge — are stated in Section 9.

9. Limitations and Threats to Validity

The shared pool is our design.

CodeBench gives each logged-in student a private container, so on the real platform these logs do not identify a shared judging queue [16]: we use them as a demand trace, with reconstructed arrivals and recorded execution durations, while the server pool, the queue and the scheduler are ours, and nothing here is a statement about waiting on the real platform. What supports the design is that a fixed pool of judge hosts on one queue is a standard deployment [2,3] and that pooling replaces 1,636 concurrent containers with 14 servers on the same demand [91]. Section 7 and Supplementary Section S2.3 give that evidence and say why the container ratio belongs to the overlay and to no deployment and is not a cost saving, the container also serving the student's editing session; Supplementary Section S6.2 says why the physical timed-work run, whose rule-conformance checks coexist with a worse guarded p99 and large per-job deviations from the ideal simulator, does not validate transport of submitted-program costs.

Open-loop replay.

Arrivals come from the log at their recorded instants and do not react to our scheduler, so a scheduler is neither rewarded with the work it would attract nor punished by the users it would drive away [92,93]. One component of that feedback is in the logs, which identify each submission's author and its position in that student's sequence: the offset between one result and the next submission is a recorded think time, non-negative for 98.38% of consecutive pairs, with median 123.5 s and Spearman correlation 0.015 with the previous job's cost. Releasing each submission after its predecessor's result plus that offset shortens every wait and reverses the benefit in none of the nine development cells: on their original-clock scores Guard(600) closes 0.734, 0.827 and 0.800 of the FCFS-to-SJF deadline-window gap at the three loads under the recorded instants and 0.825, 0.809 and 0.810 under gating (Supplementary Section S6.5). It does not restore abandonment or content feedback, and the recorded think times come from a platform on which no student ever waited, so the direction of what is left is still not identified: the results are conditional on fixed offered work rather than lower bounds on closed-loop benefit, and the exact feedback counterexample in Supplementary Section S6.3 shows a scheduling advantage disappearing in one construction and reversing in another. That section also states Feitelson's objection in his own words and the productivity distinction Shmueli and Feitelson draw. Replaying into a differently sized pool is established practice [94], and the width coupling that distorts parallel-job replays does not arise here, a job occupying one server whatever k is. One choice is deliberate: we raise load by superposing whole class-semesters aligned on weekday and clock time, rather than compressing inter-arrival times, which shrinks the daily load cycle along with the gaps [92].

The guarantee is denominated in L .

Every constant in Section 6 is a multiple of the per-job time limit. The additive constant $(3 - 2/k)L$ is an exact supremum and no artefact of the proof, it grows with k from L at a single server towards $3L$, and a promise that is to hold around every base policy cannot be less than L unless the wrapper gives up reordering altogether: whenever the budget is positive for the waiting jobs at the first dispatch epoch there are a base policy and an input on which some job's excess is at least L , and the only budget rules of this family that escape are those that make the wrapper first-come first-served itself (Proposition 3). So no mechanism of this family that reorders at all can promise less than $\Omega(L)$. The obvious escape from the $\Omega(L)$ floor is kill-and-restart tiering, and it fails. It is TAGS [95] with the dedicated hosts removed, and removing them is what breaks it: the kill becomes terminal, so the excess it leaves is wasted work rather than overtaking and no budget removes it, while the light-job constant does not improve either, a second-tier piece being still a non-preemptive block of length L . Supplementary Section S7.5 gives both arguments with the constant we measure.

Two applicability conditions, each with traces that fail it.

The first asks whether the uniform certificate is informative at the waits that matter, and four of the original six workloads fail it: their first-come first-served p99 wait runs from 0.021 to 1.78 multiples of L , against the additive term of about $3L$ in the certificate. The Intel Netbatch compute farm [18,19] is the extreme, with a service cap of 5.7 days against a worst-window p99 wait of 18.7 hours, so that the uniform additive term is about 17 days, the guard never fires at any positive budget tested, and predicted-cost ordering makes the p99 wait worse than first-come first-served there under sustained overload. This does not establish impossibility below that term for a restricted base policy or workload, and the guard results we report on the four workloads that fail the condition measure the ordering mechanism there. The second is an empirical opportunity screen and not a theorem assumption or a necessary condition for a benefit: on the two Firefox CI hardware pools [20,21] the longest 1% of jobs carry 3.4% and 5.2% of the work, at coefficients of variation 0.61 and 0.87, and there even true-size ordering makes the p99 wait worse than first-come first-served on three of the four windows we ran. Section 8.8 gives both conditions in numbers, trace by trace.

The guarantee is per job, not per quantile.

On a fixed job subset, the pointwise bound implies $Q_\alpha(W_P) \leq Q_\alpha(W_{\text{FCFS}}) + G$ by monotonicity of quantiles. It does not imply an improvement in that percentile. A guarded run can have a worse deadline-window p99 than the baseline its guarantee is stated against, by as much as the promise itself, and Supplementary Table S7 shows it doing so under two of the three corrupted scores at the busiest load while the worst single job stays inside the promise.

Development data, and what is still sealed.

Every CodeBench number in this paper is development data, and none of it is out of sample: the guard family and an early parameter grid were explored on the same overlay the tables report, before the three pre-stated grids were searched on validation overlays that exclude the evaluation semester, so the selection is validation-only inside a family that is not. The rule is a worst-cell rule and is sensitive to which validation cells it sees — at the middle promise, dropping the one cell that decides the choice moves it to a different point of the same grid — and the five overlays are week-shifts of the same class-terms rather than five independent tests, so we do not describe any of these parameters as val-

idated. The guarantee does not depend on the parameters being well chosen; only the benefit does. The sealed semesters will be opened once, after the method, the parameters and the primary metric have been frozen, and their numbers reported separately; that evaluation is not completed here, and the second judging platform's sealed id block and the corroborating trace's sealed year stay shut, the frozen pipeline covering the main trace alone (Section 7.6).

The budget shape is selected on one set of cells, and does not always transfer.

The promise holds for any budget under the cap, so nothing about the guarantee depends on the shape; what the shape decides is how much of the base policy's benefit survives, and that we select on 15 multi-server validation cells, which are the whole evidence we have for it. The single-server configuration shows the limit of that evidence: at $G = 1200$ s the queue-length shape wins the joint comparison on those cells and closes 0.701 of the gap at $k = 1$, where the age-relative shape closes 0.895, while at the two tighter promises the same choice does transfer. We report the number the pre-stated rule produces and do not re-tune on the configuration that exposes it; an operator whose pool is much smaller than the validation pool should re-run the selection rather than take our shape. The three shapes also differ in where the harm falls, so which is preferable depends on a judgement about jobs first-come first-served would have started at once, and that judgement is the operator's.

What the recorded costs mean.

The cost we replay is the platform's recorded execution time, a wall-clock duration for one evaluation that includes interpreter start-up, and part of it is contention we then re-impose inside the pool: between 38% and 79% of jobs longer than a second end in the same second as another block of that student. The observed 60-second regime is not documentation of an enforced timeout, and how the platform enforces such a limit is undocumented in the sources we found; we wrote to the maintainers about both points and had no answer. ACcoding [17] records no submission timestamps, so its arrival process is synthetic. The Firefox CI checks fit an effective capacity and a setup term on one chronological part of each window and test on the other; out of sample the mean wait, the per-hour profile and the per-job ordering survive and the 90th percentile does not, so the simulated distribution is too steep in its upper middle. Supplementary Section S2.2 gives that refit with its sharpness and circularity checks and the window lengths the split is taken inside.

History availability and the recorded-queue counterexample.

Original-clock scores may use outcomes that are still pending in the replay, and the conservative headline reduces history and changes fitted models as well as release delay, so its difference from the original score does not identify a queue-delay effect alone; the pending exact variant certifies availability for its final same-copy replay, not an online fixed point or a deployment with globally shared copied-user state (Section 8.3).

LPC-EGEE provides both recorded waits and concentrated costs, but the fitted FCFS approximation underestimates chronological-test mean waits by 24% and 39% on its two physical partitions, and with its six documented walltime classes pooled under the largest limit the work-budget guards never fire at the tested promises. Read as one pool per class, the 900-second class is the one reading of this log on which a guard does fire, at 1.42% of dispatch epochs, and it nearly spends its bound, 3,129 s of an allowed 3,150 s; that class is a pool we impose, its capacity is the boundary value of the search grid and its held-out wait profile is not reproduced, so it is not a deployment result either. We therefore treat its policy replays as conditional counterfactuals rather than validated estimates of deployment benefit; Supplementary Sections S6.4 and S6.6 give the correlations, what the

archive says about its ordering, the concurrency ceilings and middleware the replay omits, and the per-class reading with the six objections it does not answer.

Not studied.

Preemption is outside the model, so nothing here applies to a system that can checkpoint or suspend. The wrapper applies unchanged to a learned or reinforcement-learned base policy, its bound assuming nothing about the base, but we did not test one. Capacity is fixed within each replay, and the capacity sweep is a sensitivity to that choice rather than an estimate of the original platform's capacity or a capacity-planning study. The reported runs compare policies at a common set of enqueue instants and do not charge the cost of computing features and running the predictor; Section 4 measures that cost and charges it in a sensitivity, where it moves the closed gap by less than 0.001 at every load, so what is missing is not the latency but the rest of an end-to-end system. The local timed-work service was executed, but there was no prospective production deployment or measurement of user feedback. A deployed serverless scheduler of the kind this wrapper is meant for already exists, ALPS [72] being a mechanism without a proved per-job bound.

10. Conclusions

We asked whether a non-preemptive shared execution service can be reordered by predicted cost without exposing any single job to an unbounded wait. What the evidence supports is that work is the currency the excess is written in. A job's delay relative to first-come first-served equals its net overtaken work divided by the number of servers, up to a constant multiple of the time limit, and a budget on that quantity is sufficient and necessary for a per-job guarantee. Counting overtakes is not necessary for such a guarantee, and buys one only at the worst-case charge of the time limit per pass, which on our traces overstates the work that actually passes by one to two orders of magnitude. The wrapper built on it holds every job to a promise the operator states in seconds, whatever the predictor does. Under conservative visibility, Guard(600) closes 0.429 of the reference gap at the busiest load, against 0.670 for unguarded ordering. These are development results conditional on the constructed demand and information protocol. Four of the original six workloads have tail waits too short relative to L for the uniform certificate to be useful; concentration of job costs is an empirical opportunity for reordering, not a necessity claim.

Two questions are left open. The identity is stated in the dispatch-sequence currency, where the constant is exactly determined in both directions; under the executed-work convention the upward constant is unchanged and the downward one is known only to lie in $((k-1)L, 2(k-1)L]$, and closing it would make the two conventions interchangeable. And Algorithm 1 is not claimed to be the least restrictive rule carrying its guarantee: whether some wrapper overrides the base policy strictly less often and still promises G is a question we have not settled in either direction.

Supplementary Materials: The following supporting information can be downloaded at <https://www.mdpi.com/article/10.3390/math1010000/s1>. Section S1, cost prediction: the visibility audit of the feature pipeline in full; Table S1, predictors on the four non-judging traces by feature group; the release delay treated as a sensitivity; the three predictor families and the ablation ladder; Table S2, predictors on the two judging traces; and the measured cost of predicting. Section S2, traces: the field audit of the main judging trace with the release-statistics reconciliation and the two truncated archives; the capacity-sharpness and circularity checks of the continuous-integration refit; what supports the shared-pool design; and Table S3, cost concentration with the provenance of the analysis bound on every trace. Section S3, experiments: Table S4, the experimental configuration and the selection protocol; Table S5, the selected budget parameters at the three promises with the nine per-shape grid winners; Table S6, the age-relative and queue-length ablation rows of Table 5;

Table S7, the corrupted scores; Table S8, the residual of Theorem 1 on all nine cells; Table S9, the single-server configuration; Tables S10–S12, the aging comparator and the package predictor metrics; Table S13, the alternative ranking losses; Table S14, the trace-construction sensitivities; and Figures S1–S3. Section S4, the deferred proofs: Lemma 1 with its strictness, Theorem 1 with the two directions of its extremes, and Proposition 1 with the two counterexamples to its converse, together with the remaining proofs and remarks deferred from Section 6. Section S5, the reproducibility specification, including exact grids, splits, information clocks and table provenance. Section S6, the capacity sweep, physical timed-work experiment, feedback counterexamples, the closed-loop replay with recorded think times in Table S18, and the LPC-EGEE counterfactual pooled and, in Tables S19 and S20, per walltime class. Section S7, additional theory, with Table S21 comparing the nearest guarantee disciplines and reservation-and-refund charging under enforced run-time caps. Section S8, the extremal constructions, each opening with its own idea and then given in full with the tables of exact values it attains, together with the achievable fraction at one server. Section S9, the extended related work: the per-paper detail behind Section 2, every work named there being cited in the manuscript as well.

Author Contributions: [to be completed by the authors]. The template expects the CRediT roles (conceptualization, methodology, software, validation, formal analysis, investigation, data curation, writing—original draft preparation, writing—review and editing, visualization, supervision, project administration, funding acquisition), each attributed by author initials, followed by “All authors have read and agreed to the published version of the manuscript.”

Funding: [to be completed by the authors]. If there is no external funding, the template asks for “This research received no external funding”; otherwise the grant number and the funding body go here.

Institutional Review Board Statement: Not applicable.

Informed Consent Statement: Not applicable.

Data Availability Statement: The public source datasets used here are available at the locations cited, and we redistribute none of them. The CodeBench judging logs are published by the Universidade Federal do Amazonas [16]; the ACcoding dataset is published with its data descriptor [17]; the Open University Learning Analytics dataset is published with its own [87]; the Azure Functions 2021 invocation trace is published in the Azure Public Dataset repository [7], which asks that [8] be cited; and the Intel Netbatch logs are published in the Parallel Workloads Archive [18], which asks that [19] be cited and that the logs be acknowledged to their depositors. The LPC-EGEE log is also published in the Parallel Workloads Archive [90], with acknowledgment requested for its data provider. The continuous-integration trace of the two Firefox hardware worker pools, was collected by us from Mozilla’s public, unauthenticated Taskcluster and Treeherder APIs [20,21]; we do not redistribute it, and the collection scripts that rebuild it from those APIs are part of the released code. The Azure invocation trace is distributed under the publisher’s Creative Commons Attribution licence [7]; public availability does not imply an absence of licence conditions. No restricted access agreement was used, and no personal data were processed beyond the pseudonymous user identifiers the published judging datasets already carry. The simulator, the predictors, the guard implementation, configurations and archived development analyses are to be released with the paper. Supplementary Section S5 distinguishes the tables reproduced by the package from the earlier development evidence. The release will be available at [repository URL to be completed by the authors].

Acknowledgments: [to be completed by the authors].

Conflicts of Interest: [to be completed by the authors]. The template asks for “The authors declare no conflicts of interest” where that holds, and otherwise for the competing interest and for a statement on the role of any funder in the design, collection, analysis or interpretation, in the writing, or in the decision to publish.

Abbreviations

The following abbreviations are used in this manuscript:

AUPRC	Area under the precision–recall curve
AUROC	Area under the receiver operating characteristic curve
CI	Continuous integration
FCFS	First-come first-served
GRU	Gated recurrent unit
MLP	Multilayer perceptron
OULAD	Open University Learning Analytics Dataset
RMSE	Root mean squared error
SJF	Shortest job first
SPJF	Shortest predicted job first
SRPT	Shortest remaining processing time

References

- Wasik, S.; Antczak, M.; Badura, J.; Laskowski, A.; Sternal, T. A Survey on Online Judge Systems and Their Applications. *ACM Computing Surveys* **2018**, *51*, 1–34. <https://doi.org/10.1145/3143560>.
- DOMjudge team. DOMjudge Administrator’s Manual: Overview and Judging. <https://www.domjudge.org/docs/manual/main/overview.html>. Accessed 2026-09-19.
- Peveler, M.; Maicus, E.; Cutler, B. Comparing Jailed Sandboxes vs Containers Within an Autograding System. In Proceedings of the Proceedings of the 50th ACM Technical Symposium on Computer Science Education (SIGCSE ’19), 2019, pp. 139–145. <https://doi.org/10.1145/3287324.3287507>.
- König, C.J.; Kleinmann, M. Deadline Rush: A Time Management Phenomenon and Its Mathematical Description. *The Journal of Psychology* **2005**, *139*, 33–45. <https://doi.org/10.3200/JRLP.139.1.33-45>.
- Alfi, V.; Parisi, G.; Pietronero, L. Conference registration: how people react to a deadline. *Nature Physics* **2007**, *3*, 746. <https://doi.org/10.1038/nphys761>.
- Alfi, V.; Gabrielli, A.; Pietronero, L. How people react to a deadline: time distribution of conference registrations and fee payments. *Open Physics* **2009**, *7*, 483–489. <https://doi.org/10.2478/s11534-009-0059-z>.
- Microsoft Azure. Azure Functions Invocation Trace 2021. <https://github.com/Azure/AzurePublicDataset/blob/master/AzureFunctionsInvocationTrace2021.md>, 2021.
- Zhang, Y.; Goiri, Í.; Chaudhry, G.I.; Fonseca, R.; Elnikety, S.; Delimitrou, C.; Bianchini, R. Faster and Cheaper Serverless Computing on Harvested Resources. In Proceedings of the Proceedings of the ACM SIGOPS 28th Symposium on Operating Systems Principles (SOSP ’21), 2021, pp. 724–739. <https://doi.org/10.1145/3477132.3483580>.
- Ke, G.; Meng, Q.; Finley, T.; Wang, T.; Chen, W.; Ma, W.; Ye, Q.; Liu, T.Y. LightGBM: A Highly Efficient Gradient Boosting Decision Tree. In Proceedings of the Advances in Neural Information Processing Systems 30 (NeurIPS 2017), 2017.
- Schlichtkrull, M.; Kipf, T.N.; Bloem, P.; van den Berg, R.; Titov, I.; Welling, M. Modeling Relational Data with Graph Convolutional Networks. In Proceedings of the The Semantic Web (ESWC 2018), 2018, Vol. 10843, *Lecture Notes in Computer Science*, pp. 593–607. Preprint arXiv:1703.06103, https://doi.org/10.1007/978-3-319-93417-4_38.
- Scully, Z.; Grosz, I.; Mitzenmacher, M. Uniform Bounds for Scheduling with Job Size Estimates. In Proceedings of the 13th Innovations in Theoretical Computer Science Conference (ITCS 2022). Schloss Dagstuhl – Leibniz-Zentrum für Informatik, 2022, Vol. 215, *LIPIcs*, pp. 114:1–114:30. Preprint arXiv:2110.00633, <https://doi.org/10.4230/LIPIcs.ITCS.2022.114>.
- Mitzenmacher, M.; Shahout, R. Queueing, Predictions, and Large Language Models: Challenges and Open Problems. *Stochastic Systems* **2025**. Preprint arXiv:2503.07545, <https://doi.org/10.1287/stsy.2025.0106>.
- Zhao, T.; Xia, C.; Chang, X.; Li, C.; Li, W.; Zomaya, A. Competitive Fair Scheduling with Predictions. In Proceedings of the The Thirteenth International Conference on Learning Representations (ICLR 2025), 2025.
- Yu, G.; Scully, Z. Strongly Tail-Optimal Scheduling in the Light-Tailed M/G/1. *Proceedings of the ACM on Measurement and Analysis of Computing Systems* **2024**, *8*, 1–33. Preprint arXiv:2404.08826, <https://doi.org/10.1145/3656011>.
- Yu, G.; Harlev, A.; Adakroy, R.; Scully, Z. A Tale of Two Traffics: Optimizing Tail Latency in the Light-Tailed M/G/k. *Proceedings of the ACM on Measurement and Analysis of Computing Systems* **2025**, *9*, 1–40. <https://doi.org/10.1145/3771561>.
- Universidade Federal do Amazonas, Instituto de Computação. CodeBench Dataset. <https://codebench.icomp.ufam.edu.br/dataset/>. Accessed 2026-09-19.

17. Chen, K.; Huang, F.; Liu, Z.; Yu, H.; Meng, L.; Mo, S.; Zhang, L.; Song, Y. ACcoding: A graph-based dataset for online judge programming. *Scientific Data* **2024**, *11*. See also Author Correction, doi:10.1038/s41597-024-03693-3, <https://doi.org/10.1038/s41597-024-03392-z>. 1565
18. Parallel Workloads Archive. The Intel Netbatch Logs. https://www.cs.huji.ac.il/labs/parallel/workload/I_intel_netbatch/, 2012. 1566
19. Shai, O.; Shmueli, E.; Feitelson, D.G. Heuristics for Resource Matching in Intel's Compute Farm. In Proceedings of the Job Scheduling Strategies for Parallel Processing (JSSPP 2013). Springer, 2014, Lecture Notes in Computer Science, pp. 116–135. https://doi.org/10.1007/978-3-662-43779-7_7. 1567
20. Mozilla. Taskcluster Documentation: Queue Service Reference. <https://docs.taskcluster.net/docs/reference/platform/queue>. Accessed 2026-09-19. 1568
21. Mozilla. Treeherder REST API Documentation. <https://treeherder.mozilla.org/docs/>. Accessed 2026-09-19. 1569
22. Purohit, M.; Svitkina, Z.; Kumar, R. Improving Online Algorithms via ML Predictions. In Proceedings of the Advances in Neural Information Processing Systems 31 (NeurIPS 2018), 2018. 1570
23. Mitzenmacher, M. Scheduling with Predictions and the Price of Misprediction. In Proceedings of the 11th Innovations in Theoretical Computer Science Conference (ITCS 2020). Schloss Dagstuhl – Leibniz-Zentrum für Informatik, 2020, Vol. 151, *LIPICs*, pp. 14:1–14:18. Preprint arXiv:1902.00732, <https://doi.org/10.4230/LIPICs.ITCS.2020.14>. 1571
24. Azar, Y.; Leonardi, S.; Touitou, N. Flow Time Scheduling with Uncertain Processing Time. In Proceedings of the Proceedings of the 53rd Annual ACM SIGACT Symposium on Theory of Computing (STOC), 2021, pp. 1070–1080. 1572
25. Azar, Y.; Leonardi, S.; Touitou, N. Distortion-Oblivious Algorithms for Minimizing Flow Time. In Proceedings of the Proceedings of the 2022 Annual ACM-SIAM Symposium on Discrete Algorithms (SODA), 2022, pp. 252–274. Preprint arXiv:2109.08424. 1573
26. Dong, J.; Ibrahim, R. Shortest-Job-First Scheduling in Many-Server Queues with Impatient Customers and Noisy Service-Time Estimates. *Operations Research* **2025**, *73*, 3139–3155. <https://doi.org/10.1287/opre.2022.310>. 1574
27. Shahout, R.; Mitzenmacher, M. SkipPredict: When to Invest in Predictions for Scheduling. In Proceedings of the Advances in Neural Information Processing Systems 37 (NeurIPS 2024), 2024, pp. 105663–105701. Preprint arXiv:2402.03564, <https://doi.org/10.52202/079017-3352>. 1575
28. Schwiegelshohn, U.; Yahyapour, R. Fairness in Parallel Job Scheduling. *Journal of Scheduling* **2000**, *3*, 297–320. [https://doi.org/10.1002/1099-1425\(200009/10\)3:5<297::AID-JOS50>3.0.CO;2-D](https://doi.org/10.1002/1099-1425(200009/10)3:5<297::AID-JOS50>3.0.CO;2-D). 1576
29. Sabin, G.; Kochhar, G.; Sadayappan, P. Job Fairness in Non-Preemptive Job Scheduling. In Proceedings of the Proceedings of the 2004 International Conference on Parallel Processing (ICPP 2004). IEEE Computer Society, 2004, pp. 186–194. <https://doi.org/10.1109/ICPP.2004.1327920>. 1577
30. Sabin, G.; Sadayappan, P. Unfairness Metrics for Space-Sharing Parallel Job Schedulers. In Proceedings of the Job Scheduling Strategies for Parallel Processing (JSSPP 2005), 2005, Vol. 3834, *Lecture Notes in Computer Science*, pp. 238–256. https://doi.org/10.1007/11605300_12. 1578
31. Rajbhandary, A.; Bunde, D.P.; Leung, V.J. Variations of Conservative Backfilling to Improve Fairness. In Proceedings of the Job Scheduling Strategies for Parallel Processing (JSSPP 2013). Springer, 2014, Vol. 8429, *Lecture Notes in Computer Science*, pp. 177–191. https://doi.org/10.1007/978-3-662-43779-7_10. 1579
32. Lindsay, A.M.; Galloway-Carson, M.; Johnson, C.R.; Bunde, D.P.; Leung, V.J. Backfilling with guarantees made as jobs arrive. *Concurrency and Computation: Practice and Experience* **2013**, *25*, 513–523. <https://doi.org/10.1002/cpe.2860>. 1580
33. Yuan, Y.; Yang, G.; Wu, Y.; Zheng, W. PV-EASY: A Strict Fairness Guaranteed and Prediction Enabled Scheduler in Parallel Job Scheduling. In Proceedings of the Proceedings of the 19th ACM International Symposium on High Performance Distributed Computing (HPDC '10), 2010, pp. 240–251. <https://doi.org/10.1145/1851476.1851505>. 1581
34. Yuan, Y.; Wu, Y.; Zheng, W.; Li, K. Guarantee Strict Fairness and Utilize Prediction Better in Parallel Job Scheduling. *IEEE Transactions on Parallel and Distributed Systems* **2014**, *25*, 971–981. <https://doi.org/10.1109/TPDS.2013.88>. 1582
35. Grosz, I.; Scully, Z.; Harchol-Balter, M. Load Balancing Guardrails: Keeping Your Heavy Traffic on the Road to Low Response Times. arXiv preprint arXiv:1905.03439, 2019, [arXiv:cs.PF/1905.03439]. <https://doi.org/10.48550/arXiv.1905.03439>. 1583
36. Ramadge, P.J.; Wonham, W.M. Supervisory Control of a Class of Discrete Event Processes. *SIAM Journal on Control and Optimization* **1987**, *25*, 206–230. <https://doi.org/10.1137/0325013>. 1584
37. Grosz, I.; Yang, K.; Scully, Z.; Harchol-Balter, M. Nudge: Stochastically Improving upon FCFS. *Proceedings of the ACM on Measurement and Analysis of Computing Systems* **2021**, *5*, 1–29. Preprint arXiv:2106.01492, <https://doi.org/10.1145/3460088>. 1585
38. Van Houdt, B. On the Stochastic and Asymptotic Improvement of First-Come First-Served and Nudge Scheduling. *Proceedings of the ACM on Measurement and Analysis of Computing Systems* **2022**, *6*, 1–22. Preprint arXiv:2206.10428, <https://doi.org/10.1145/3570610>. 1586
39. Charlet, N.; Van Houdt, B. Tail Optimality and Performance Analysis of the Nudge*(M) Scheduling Algorithm. *Operations Research* **2026**, *74*, 2161–2178. <https://doi.org/10.1287/opre.2024.1061>. 1587

40. Grosz, I.; Harchol-Balter, M.; Scheller-Wolf, A. WCFS: a new framework for analyzing multiserver systems. *Queueing Systems* **2022**, *102*, 143–174. Preprint arXiv:2109.12663, <https://doi.org/10.1007/s11134-022-09848-6>. 1619
41. Bennett, J.C.R.; Zhang, H. WF²Q: Worst-case Fair Weighted Fair Queueing. In Proceedings of the Proceedings of IEEE INFOCOM '96, 1996, Vol. 1, pp. 120–128. <https://doi.org/10.1109/INFCOM.1996.497885>. 1620
42. Parekh, A.K.; Gallager, R.G. A generalized processor sharing approach to flow control in integrated services networks: the single-node case. *IEEE/ACM Transactions on Networking* **1993**, *1*, 344–357. <https://doi.org/10.1109/90.234856>. 1621
43. Parekh, A.K.; Gallager, R.G. A generalized processor sharing approach to flow control in integrated services networks: the multiple node case. *IEEE/ACM Transactions on Networking* **1994**, *2*, 137–150. <https://doi.org/10.1109/90.298432>. 1622
44. Stoica, I.; Abdel-Wahab, H. Earliest Eligible Virtual Deadline First: A Flexible and Accurate Mechanism for Proportional Share Resource Allocation. Technical Report TR-95-22, Department of Computer Science, Old Dominion University, Norfolk, Virginia, 1995. Revised 26 January 1996. 1623
45. Bennett, J.C.R.; Benson, K.; Charny, A.; Courtney, W.F.; Le Boudec, J.Y. Delay Jitter Bounds and Packet Scale Rate Guarantee for Expedited Forwarding. *IEEE/ACM Transactions on Networking* **2002**, *10*, 529–540. <https://doi.org/10.1109/TNET.2002.801404>. 1624
46. Graham, R.L. Bounds on Multiprocessing Timing Anomalies. *SIAM Journal on Applied Mathematics* **1969**, *17*, 416–429. <https://doi.org/10.1137/0117039>. 1625
47. Jeffay, K.; Stanat, D.F.; Martel, C.U. On Non-Preemptive Scheduling of Periodic and Sporadic Tasks. In Proceedings of the Proceedings of the 12th IEEE Real-Time Systems Symposium (RTSS 1991), 1991, pp. 129–139. <https://doi.org/10.1109/REAL.1991.160366>. 1626
48. Kleinrock, L. A conservation law for a wide class of queueing disciplines. *Naval Research Logistics Quarterly* **1965**, *12*, 181–192. <https://doi.org/10.1002/nav.3800120206>. 1627
49. Green, T.C.; Stidham, Jr., S. Sample-path conservation laws, with applications to scheduling queues and fluid systems. *Queueing Systems* **2000**, *36*, 175–199. <https://doi.org/10.1023/A:1019183220080>. 1628
50. El-Taha, M. A general workload conservation law with applications to queueing systems. *Queueing Systems* **2017**, *85*, 361–381. <https://doi.org/10.1007/s11134-017-9515-4>. 1629
51. Bartsch, B.; Bolch, G. A conservation law for G/G/m queueing systems. *Acta Informatica* **1978**, *10*, 105–109. <https://doi.org/10.1007/BF00260928>. 1630
52. Federgruen, A.; Groenevelt, H. M/G/c Queueing Systems with Multiple Customer Classes: Characterization and Control of Achievable Performance Under Nonpreemptive Priority Rules. *Management Science* **1988**, *34*, 1121–1138. <https://doi.org/10.1287/mnsc.34.9.1121>. 1631
53. Grosz, I.; Scully, Z.; Harchol-Balter, M. SRPT for multiserver systems. *Performance Evaluation* **2018**, *127–128*, 154–175. Preprint arXiv:1805.07686, <https://doi.org/10.1016/j.peva.2018.10.001>. 1632
54. Kleinrock, L. A delay dependent queue discipline. *Naval Research Logistics Quarterly* **1964**, *11*, 329–341. <https://doi.org/10.1002/nav.3800110306>. 1633
55. Stanford, D.A.; Taylor, P.; Ziedins, I. Waiting time distributions in the accumulating priority queue. *Queueing Systems* **2014**, *77*, 297–330. Online first 2013-12-07, <https://doi.org/10.1007/s11134-013-9382-6>. 1634
56. Bansal, N.; Harchol-Balter, M. Analysis of SRPT scheduling. In Proceedings of the Proceedings of the 2001 ACM SIGMETRICS International Conference on Measurement and Modeling of Computer Systems, 2001, pp. 279–290. <https://doi.org/10.1145/378420.378792>. 1635
57. Wierman, A.; Harchol-Balter, M. Classifying scheduling policies with respect to unfairness in an M/GI/1. In Proceedings of the Proceedings of the 2003 ACM SIGMETRICS International Conference on Measurement and Modeling of Computer Systems, 2003, pp. 238–249. <https://doi.org/10.1145/781027.781057>. 1636
58. Sharif, A.B.; Stanford, D.A.; Taylor, P.; Ziedins, I. A multi-class multi-server accumulating priority queue with application to health care. *Operations Research for Health Care* **2014**, *3*, 73–79. <https://doi.org/10.1016/j.orhc.2014.01.002>. 1637
59. Li, N.; Stanford, D.A. Multi-server accumulating priority queues with heterogeneous servers. *European Journal of Operational Research* **2016**, *252*, 866–878. <https://doi.org/10.1016/j.ejor.2016.02.010>. 1638
60. Whitt, W. The amount of overtaking in a network of queues. *Networks* **1984**, *14*, 411–426. <https://doi.org/10.1002/net.3230140305>. 1639
61. Kim, W.S.; Lim, D.E. Analysis of overtaking in M/M/c queues. *Computers & Industrial Engineering* **2016**, *101*, 177–183. <https://doi.org/10.1016/j.cie.2016.09.005>. 1640
62. Kiefer, J.; Wolfowitz, J. On the theory of queues with many servers. *Transactions of the American Mathematical Society* **1955**, *78*, 1–18. <https://doi.org/10.1090/S0002-9947-1955-0066587-3>. 1641
63. Daley, D.J. Certain optimality properties of the first-come first-served discipline for G/G/s queues. *Stochastic Processes and their Applications* **1987**, *25*, 301–308. [https://doi.org/10.1016/0304-4149\(87\)90208-0](https://doi.org/10.1016/0304-4149(87)90208-0). 1642
64. Tsafir, D.; Etsion, Y.; Feitelson, D.G. Backfilling Using System-Generated Predictions Rather than User Runtime Estimates. *IEEE Transactions on Parallel and Distributed Systems* **2007**, *18*, 789–803. <https://doi.org/10.1109/TPDS.2007.70606>. 1643

65. Gaussier, E.; Glesser, D.; Reis, V.; Trystram, D. Improving backfilling by using machine learning to predict running times. In Proceedings of the Proceedings of the International Conference for High Performance Computing, Networking, Storage and Analysis (SC '15), 2015, pp. 1–10. <https://doi.org/10.1145/2807591.2807646>. 1674
66. Gaussier, E.; Lelong, J.; Reis, V.; Trystram, D. Online Tuning of EASY-Backfilling using Queue Reordering Policies. *IEEE Transactions on Parallel and Distributed Systems* **2018**, *29*, 2304–2316. <https://doi.org/10.1109/TPDS.2018.2820699>. 1675
67. Zhang, E.; Wu, F.; Lu, X. An Intelligent Scheduling System for Large-Scale Online Judging. In Proceedings of the Computer Science and Education. Computer Science and Technology (ICCSE 2023). Springer Nature Singapore, 2024, Vol. 2023, *Communications in Computer and Information Science*, pp. 265–279. https://doi.org/10.1007/978-981-97-0730-0_24. 1676
68. Fu, Y.; Zhu, S.; Su, R.; Qiao, A.; Stoica, I.; Zhang, H. Efficient LLM Scheduling by Learning to Rank, 2024, [2408.15792]. 1677
69. Li, Y.; Chen, Y.; Chen, J.; Choukse, E.; Qiu, H.; Suh, G.E.; Fonseca, R.; Scully, Z.; Gupta, U. Beyond Prediction: Tail-Aware Scheduling for LLM Inference, 2026, [arXiv:cs.LG/2606.18431]. <https://doi.org/10.48550/arXiv.2606.18431>. 1678
70. Luo, Z.; Liu, J.; Lee, M.; Shroff, N.B. Prediction-Assisted Online Distributed Deep Learning Workload Scheduling in GPU Clusters. In Proceedings of the IEEE INFOCOM 2025 – IEEE Conference on Computer Communications, 2025, pp. 1–10. Preprint arXiv:2501.05563, <https://doi.org/10.1109/INFOCOM55648.2025.11044761>. 1679
71. Fu, Y.; Liu, L.; Wang, H.; Cheng, Y.; Chen, S. SFS: Smart OS Scheduling for Serverless Functions. In Proceedings of the SC22: International Conference for High Performance Computing, Networking, Storage and Analysis. IEEE, 2022, pp. 1–16. Preprint arXiv:2209.01709, <https://doi.org/10.1109/SC41404.2022.00047>. 1680
72. Fu, Y.; Shi, R.; Wang, H.; Chen, S.; Cheng, Y. ALPS: An Adaptive Learning, Priority OS Scheduler for Serverless Functions. In Proceedings of the 2024 USENIX Annual Technical Conference (USENIX ATC 24). USENIX Association, 2024, pp. 19–36. 1681
73. Massey, W.A.; Whitt, W. Peak congestion in multi-server service systems with slowly varying arrival rates. *Queueing Systems* **1997**, *25*, 157–172. <https://doi.org/10.1023/A:1019156418862>. 1682
74. Jennings, O.B.; Mandelbaum, A.; Massey, W.A.; Whitt, W. Server Staffing to Meet Time-Varying Demand. *Management Science* **1996**, *42*, 1383–1394. <https://doi.org/10.1287/mnsc.42.10.1383>. 1683
75. Green, L.V.; Kolesar, P.J.; Whitt, W. Coping with Time-Varying Demand When Setting Staffing Requirements for a Service System. *Production and Operations Management* **2007**, *16*, 13–39. <https://doi.org/10.1111/j.1937-5956.2007.tb00164.x>. 1684
76. Feldman, Z.; Mandelbaum, A.; Massey, W.A.; Whitt, W. Staffing of Time-Varying Queues to Achieve Time-Stable Performance. *Management Science* **2008**, *54*, 324–338. <https://doi.org/10.1287/mnsc.1070.0821>. 1685
77. Harchol-Balter, M. *Performance Modeling and Design of Computer Systems: Queueing Theory in Action*; Cambridge University Press, 2013. <https://doi.org/10.1017/CBO9781139226424>. 1686
78. Xu, K.; Hu, W.; Leskovec, J.; Jegelka, S. How Powerful are Graph Neural Networks? In Proceedings of the International Conference on Learning Representations (ICLR 2019), 2019. Preprint arXiv:1810.00826. 1687
79. Robinson, J.; Ranjan, R.; Hu, W.; Huang, K.; Han, J.; Dobles, A.; Fey, M.; Lenssen, J.E.; Yuan, Y.; Zhang, Z.; et al. RelBench: A Benchmark for Deep Learning on Relational Databases. In Proceedings of the Advances in Neural Information Processing Systems 37, Datasets and Benchmarks Track (NeurIPS 2024), 2024. Preprint arXiv:2407.20060. 1688
80. Gu, J.; Ranjan, R.; Kanatsoulis, C.; Tang, H.; Jurkovic, M.; Hudovernik, V.; Znidar, M.; Chaturvedi, P.; Shroff, P.; Li, F.; et al. RelBench v2: A Large-Scale Benchmark and Repository for Relational Data, 2026, [2602.12606]. 1689
81. Grinsztajn, L.; Oyallon, E.; Varoquaux, G. Why Do Tree-Based Models Still Outperform Deep Learning on Typical Tabular Data? In Proceedings of the Advances in Neural Information Processing Systems 35 (NeurIPS 2022), 2022, pp. 507–520. <https://doi.org/10.52202/068431-0037>. 1690
82. Jørgensen, B. Exponential Dispersion Models. *Journal of the Royal Statistical Society, Series B* **1987**, *49*, 127–145. <https://doi.org/10.1111/j.2517-6161.1987.tb01685.x>. 1691
83. Yang, Y.; Qian, W.; Zou, H. Insurance Premium Prediction via Gradient Tree-Boosted Tweedie Compound Poisson Models. *Journal of Business & Economic Statistics* **2018**, *36*, 456–470. <https://doi.org/10.1080/07350015.2016.1200981>. 1692
84. Duan, N. Smearing Estimate: A Nonparametric Retransformation Method. *Journal of the American Statistical Association* **1983**, *78*, 605–610. <https://doi.org/10.1080/01621459.1983.10478017>. 1693
85. Mu'alem, A.W.; Feitelson, D.G. Utilization, predictability, workloads, and user runtime estimates in scheduling the IBM SP2 with backfilling. *IEEE Transactions on Parallel and Distributed Systems* **2001**, *12*, 529–543. <https://doi.org/10.1109/71.932708>. 1694
86. Smith, W.E. Various optimizers for single-stage production. *Naval Research Logistics Quarterly* **1956**, *3*, 59–66. <https://doi.org/10.1002/nav.3800030106>. 1695
87. Kuzilek, J.; Hlosta, M.; Zdrahal, Z. Open University Learning Analytics dataset. *Scientific Data* **2017**, *4*, 170171. <https://doi.org/10.1038/sdata.2017.171>. 1696
88. Shahradd, M.; Fonseca, R.; Goiri, Í.; Chaudhry, G.; Batur, P.; Cooke, J.; Laureano, E.; Tresness, C.; Russinovich, M.; Bianchini, R. Serverless in the Wild: Characterizing and Optimizing the Serverless Workload at a Large Cloud Provider. In Proceedings of the 2020 USENIX Annual Technical Conference (USENIX ATC 20). USENIX Association, 2020, pp. 205–218. 1697

89. Verma, A.; Pedrosa, L.; Korupolu, M.; Oppenheimer, D.; Tune, E.; Wilkes, J. Large-scale cluster management at Google with Borg. In Proceedings of the Proceedings of the Tenth European Conference on Computer Systems (EuroSys '15), 2015, pp. 1–17. <https://doi.org/10.1145/2741948.2741964>. 1728
90. Tsafir, D. The LPC Log. https://www.cs.huji.ac.il/labs/parallel/workload/1_lpc/. Parallel Workloads Archive; data provided by Emmanuel Medernach; accessed 2026-09-23. 1729
91. Smith, D.R.; Whitt, W. Resource Sharing for Efficiency in Traffic Systems. *Bell System Technical Journal* **1981**, *60*, 39–55. <https://doi.org/10.1002/j.1538-7305.1981.tb00221.x>. 1730
92. Feitelson, D.G. Resampling with Feedback: A New Paradigm of Using Workload Data for Performance Evaluation. In *Job Scheduling Strategies for Parallel Processing (JSSPP 2021)*; Springer, 2021; Vol. 12985, *Lecture Notes in Computer Science*, pp. 3–32. https://doi.org/10.1007/978-3-030-88224-2_1. 1731
93. Shmueli, E.; Feitelson, D.G. On Simulation and Design of Parallel-Systems Schedulers: Are We Doing the Right Thing? *IEEE Transactions on Parallel and Distributed Systems* **2009**, *20*, 983–996. <https://doi.org/10.1109/TPDS.2008.152>. 1732
94. Harchol-Balter, M.; Schroeder, B.; Bansal, N.; Agrawal, M. Size-based scheduling to improve web performance. *ACM Transactions on Computer Systems* **2003**, *21*, 207–233. <https://doi.org/10.1145/762483.762486>. 1733
95. Harchol-Balter, M. Task assignment with unknown duration. *Journal of the ACM* **2002**, *49*, 260–288. Conference version: Proceedings of the 20th International Conference on Distributed Computing Systems (ICDCS 2000), pp. 214–224, doi:10.1109/ICDCS.2000.840932, <https://doi.org/10.1145/506147.506154>. 1734

Disclaimer/Publisher's Note: The statements, opinions and data contained in all publications are solely those of the individual author(s) and contributor(s) and not of MDPI and/or the editor(s). MDPI and/or the editor(s) disclaim responsibility for any injury to people or property resulting from any ideas, methods, instructions or products referred to in the content. 1735